Сумський державний університет
Кафедра
Прикладної математики та моделювання складних систем

Курсова робота
Дисципліна
Машинне навчання

Студентка: Пороскун О. О.
Група: ПМ.м-21
Викладач: Харченко В. О.

Суми, Сумська область
2023

ЗМІСТ

Вступ 3

Постановка задачі 4

Теоретична частина 5

Класифікація методом дерева прийняття рішень (decision tree) 5

Класифікація методом k-найближчих сусідів (kNN - k-Nearest Neighbors) 6

Класифікація методом опорних векторів (SVM) 9

Зменшення розмірності методом головних компонент (PCA) 11

Зменшення розмірності методом сингулярного розкладання (SVD) 15

Кластеризація методом k-середніх (k-Means) 16

Кластеризація методом DBscan 21

Пошук асоціативних правил з використанням методу Apriori 24

Програмна реалізація 28

Метод дерева прийняття рішень (decision tree) 28

Метод класифікації kNN 33

Метод опорних векторів (SVM) 40

Метод головних компонент (PCA) 51

Редукування даних: метод SVD 60

Метод k-середніх (k-Means) 65

Метод DBSCAN 79

Асоціації та пошук правил: метод Apriori 86

Висновки 92

Вступ

Перш ніж говорити про навчання, почнемо з аналізу термінології.

Data Science - це загальне найменування дисциплін з вивчення даних, Machine Learning - це підрозділ Data Science, який займається розбудовою розумних моделей. Такі моделі можуть використовуватися для передбачення покупки товару користувачем, рекомендацій у соцмережах (рекомендаційні системи), розпізнавання зображень тощо.

Data Science спеціалісти займаються дослідженнями. В іноземних компаніях такій посаді відповідають позиціїresearch-інженерів — це переважно математики, які працюють з теоретичною частиною алгоритмів та досліджують різноманітні закономірності.

Machine Learning інженери, своєю чергою, займаються побудовою моделей з урахуванням отриманих даних. Але такий поділ існує лише теоретично чи лише деяких країнах.

В Україні Data Science та Machine Learning раніше використовувалися як слова-синоніми, зараз ці поняття вже починають розділяти. У наших реаліях вакансії, де необхідне знання Machine Learning, найчастіше називаються Data Scientist і навпаки. Тому, якщо ви хочете працювати з даними, вам слід вивчити і те, й інше.

Класи застосування машинного навчання(МН)

- Навчання з учителем (Supervised learning). Ми маємо чимало прикладів і правильні, еталонні, відповіді до кожного з них. Наприклад, історичні дані успішности студентів з минулих років.
- Навчання без учителя (Unsupervised learning). Це знаходження сенсу в даних без наявности конкретної, правильної, відповіді. Інакше кажучи, ми не знаємо, що шукаємо, але дуже сильно хочемо знайти.
- Навчання із закріпленням (Reinforcement learning). Це розширений варіант навчання з учителем, коли замість еталонних відповідей програма наприкінці отримує зворотний зв’язок. Наприклад, якщо ми вчимо машину грати в шахи, оцінка кожного можливого ходу є надто затратною й не завжди правильною, але наприкінці гри дуже легко визначити виграш або програш алгоритму.
- Перенесення навчання (Transfer learning). Навчаємо модель для однієї проблеми й використовуємо результат навчання для модифікації даних від іншої. Наприклад, фільтр, за допомогою якого ви робите селфі подібними до творінь Вінсента ван Гога та який домальовує заячі вуха у відеоконференції або змушує Обаму вимовляти ваші слова його ж голосом і з його мімікою.

Мета машиного навчання — передбачити результат за вхідними даними.

Постановка задачі

Потрібно створити алгоритми або використати вбудовані функції на мові програмування Python за 8 темами машинного навчання:

1. класифікація методом дерева прийняття рішень (decision tree),
2. класифікація методом k-найближчих сусідів (kNN - k-Nearest Neighbors),
3. класифікація методом опорних векторів (SVM),
4. зменшення розмірності методом головних компонент (PCA),
5. зменшення розмірності методом сингулярного розкладання (SVD),
6. кластеризація методом k-середніх (k-Means),
7. кластеризація методом DBscan,
8. пошук асоціативних правил з використанням методу Apriori.

Теоретична частина

Класифікація методом дерева прийняття рішень (decision tree)

Дерева рішень Decision Tree є одним із найбільш ефективних інструментів інтелектуального аналізу даних та передбачуваної аналітики, які дозволяють вирішувати завдання класифікації та регресії.

Оскільки правила в деревах рішень виходять шляхом узагальнення безлічі окремих спостережень (навчальних прикладів), що описують предметну область, то за аналогією з відповідним методом логічного виведення їх називають індуктивними правилами, а процес навчання – індукцією дерев рішень.

У навчальній множині для прикладів має бути задане цільове значення, оскільки дерева рішень є моделями, що будуються на основі навчання з учителем. При цьому якщо цільова змінна дискретна (мітка класу), то модель називають деревом класифікації, а якщо безперервна, то деревом регресії.

Структура дерева рішень

Введемо до розгляду основні поняття, що використовуються в теорії дерев рішень.

Табл. 1 Основні поняття в теорії дерев рішень

Назва	Опис
Об’єкт	Приклад, шаблон, спостереження
Атрибут	Ознака, незалежна змінна, властивість
Цільова змінна	Залежна змінна, мітка класу
Вузол	Внутрішній вузол дерева, вузол перевірки
Кореневий вузол	Початковий вузол дерева рішень
Лист	Кінцевий вузол дерева, вузол рішення
Вирішальне правило	Умова у вузлі, перевірка

Власне, саме дерево рішень — метод подання вирішальних правил у ієрархічній структурі, що складається з елементів двох типів – вузлів (node) та листя (leaf). У вузлах знаходяться вирішальні правила і проводиться перевірка відповідності прикладів цього правила за яким-небудь атрибутом навчальної множини.

У найпростішому випадку, в результаті перевірки, множина прикладів, що потрапили у вузол, розбивається на дві підмножини, в одну з яких потрапляють приклади, що задовольняють правило, а в інше не задовольняють. Потім до

кожної підмножини знову застосовується правило і процедура рекурсивно повторюється доки буде досягнуто деяка умова зупинки алгоритму. В результаті в останньому вузлі перевірка та розбиття не проводиться і він оголошується листом. Лист визначає рішення для кожного прикладу, що в нього потрапив. Для дерева класифікації це клас, що асоціюється з вузлом, а для дерева регресії модальний інтервал цільової змінної, що відповідає листу.

Таким чином, на відміну від вузла, у листі міститься не правило, а підмножина об'єктів, що задовольняють всі правила гілки, яка закінчується цим листом. Очевидно, щоб потрапити в лист, приклад повинен задовольняти всі правила, що лежать на шляху до цього листа. Оскільки шлях у дереві до кожного листа єдиний, то й кожен приклад може потрапити лише в один лист, що забезпечує єдиність рішення.

Основні етапи побудови

У ході побудови дерева рішень необхідно вирішити кілька основних проблем, з кожною з яких пов'язаний відповідний крок процесу навчання:

- Вибір атрибута, за яким буде проводитися розбиття в даному вузлі (атрибута розбиття).
- Вибір критерію зупинення навчання.
- Вибір методу відсікання гілок (спрощення).
- Оцінка точності збудованого дерева.

Класифікація методом k-найближчих сусідів (kNN - k-Nearest Neighbors)

Типовим представником методів класифікації є метод k-найближчих сусідів k-nearest neighbors algorithm (KNN). Він використовує просту логіку: об'єкт відноситься до того ж класу, що більшість його найближчих сусідів.

Метод належить до класу непараметричних, тобто не вимагає припущень про те, з якого статистичного розподілу була сформована навчальна множина. Отже, класифікаційні моделі, побудовані за допомогою методу KNN, також будуть непараметричними. Це означає, що структура моделі не визначається жорстко спочатку, а визначається даними.

Оскільки ознаки, на основі яких проводиться класифікація, можуть мати різну фізичну природу і, відповідно, діапазони значень, для покращення результатів класифікації буде корисно виконати нормалізацію навчальних даних.

Алгоритм

Нехай є набір даних, що складається з n спостережень $X_i(i = 1, \ldots, n)$ для кожного з яких заданий клас C_j $(j = 1, \ldots, m)$. Тоді на його основі може бути сформована навчальна множина, всі приклади якої являють собою пари X_i, C_j .

Алгоритм KNN можна розділити на дві прості фази: навчання та класифікації. Під час навчання алгоритм просто запам'ятовує вектори ознак спостережень та його мітки класів (тобто приклади). Також задається параметр алгоритму k, який задає число "сусідів", які будуть використовуватися при класифікації.

На фазі класифікації пред'являється новий об'єкт, котрого мітка класу не задана. Для нього визначаються k найближчих (у сенсі деякої метрики) попередньо класифікованих спостережень. Потім вибирається клас, якому належить більшість з k найближчих прикладів-сусідів, і до цього ж класу відноситься об'єкт, що класифікується. Пояснимо роботу алгоритму за допомогою рисунка.

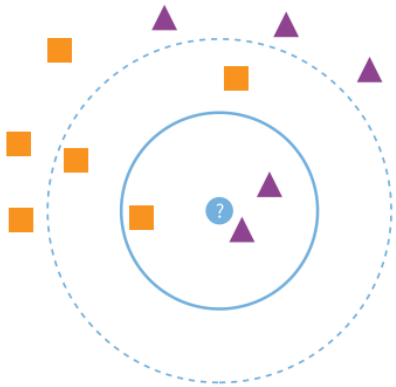


Рис. 1 Робота алгоритму KNN

Кружком представлений об'єкт, який потрібно класифікувати, віднісши до одного з двох класів «трикутники» та «квадрати».

Якщо вибрати k = 3, то з трьох найближчих об'єктів виявляться два «трикутниками» і один «квадратом». Відтак новому об'єкту буде надано клас «трикутник».

Якщо поставити k = 5, то з п'яти «сусідів» два будуть «трикутники» і три «квадрати», в результаті об'єкт, що класифікується, буде розпізнаний як «квадрат».

Визначення класу нового об'єкту

У найпростішому випадку клас нового об'єкта може бути визначений простим вибором класу, що найчастіше зустрічається серед k прикладів. При цьому розраховується відстань від даного об'єкта до кожного об'єкта з навчальної вибірки за формулою

$$D_j = \sqrt{\sum_{i=1}^{n}(x_i - a_i)^2}$$

де x_i– і-та ознака об'єкта, що класифікується a_i – ознака класифікованих об'єктів.

Однак на практиці це не завжди вдале рішення, наприклад, якщо частота появи для двох або більше класів виявляється однаковою. І тут використовують деяку функцію, з допомогою якої визначається клас, звану функцією поєднання (combination function). У звичайному випадку використовують так зване просте невиважене голосування (simple unweighted voting). При цьому передбачається, що всі k прикладів мають однакове право «голосу» незалежно від відстань до об'єкта, що класифікується. Однак, логічно припустити, що чим далі приклад розташований від об'єкта, що класифікується, в просторі ознак, тим нижче йозначимість для визначення класу. Тому для покращення результатів класифікації вводять зважування прикладів залежно від їхньої віддаленості. І тут використовують зважене голосування (weighted voting).

В основі ідеї зваженого голосування лежить введення «штрафу» для класу, залежно від того, наскільки приклади, що відносяться до нього, віддалені від класифікованого об'єкта. Такий "штраф" представляється як сума величин, обернених квадрату відстаней від прикладу j-го класу до класифікованого об'єкта (часто це значення називають показником близькості):

$$Q_i = \sum_{i=1}^{n_j} \frac{1}{D^2(x, a_{ij})}$$

де D – оператор обчислення відстані, x – вектор ознак об'єкта, що класифікується, a_{ij} – і-й приклад j-го класу.

Таким чином, «перемагає» той клас, для якого величина Qj виявиться найбільшою. При цьому також знижується ймовірність того, що класи отримають однакову кількість голосів.

Області застосування алгоритму

Алгоритм KNN може застосовуватися практично у всіх завданнях

класифікації, особливо у випадках, коли оцінити параметри ймовірнісного розподілу даних складно чи неможливо. Найбільш типовими програмами алгоритму KNN є:

• класифікація клієнтів (наприклад, за рівнем лояльності);

• медицина – класифікація пацієнтів за медичними показниками;

• маркетинг – класифікація товарів за рівнем популярності і т. д.

Переваги та недоліки алгоритму

До *переваг* алгоритму можна віднести.

• стійкість до викидів і аномальних значень, оскільки ймовірність попадання записів, що містять їх, до k-найближчих сусідів мала. Якщо ж це сталося, то вплив на голосування (особливо зважене) також, швидше за все, буде незначним, а отже, малим буде і вплив на результати класифікації;

• програмна реалізація алгоритму доволі проста;

• результати роботи алгоритму легко піддаються інтерпретації. Логіка роботи алгоритму зрозуміла експертам у різних галузях.

До *недоліків* алгоритму KNN можна віднести:

• даний метод не створює будь-яких моделей, які узагальнюють попередній досвід, а інтерес можуть становити й самі правила класифікації;

• при класифікації об’єкта використовуються всі доступні дані, тому метод KNN є досить витратним у плані, особливо у разі великих обсягів даних;

• висока трудомісткість через необхідність обчислення відстаней до всіх прикладів;

• підвищені вимоги до репрезентативності вихідних даних

Класифікація методом опорних векторів (SVM)

Розглянемо метод опорних векторів (англ. SVM, Support Vector Machine) для задач класифікації. Буде представлено основну ідею алгоритму, виведення налаштування його ваг і розібрано просту реалізацію своїми руками. На прикладі датасету Iris буде продемонстровано роботу написаного алгоритму з лінійно розділеними-/нерозділними даними у просторі R^2. Додатково будуть озвучені плюси та мінуси алгоритму, його модифікації.

Завдання

Вирішуватимемо завдання бінарної (коли класів всього два) класифікації. Спочатку алгоритм тренується на об’єктах з навчальної вибірки, котрим заздалегідь відомі мітки класів. Далі вже навчений алгоритм передбачає мітку класу для кожного об’єкта з відкладеної/тестової вибірки. Мітки класів можуть набувати значень $Y = \{-1, +1\}$. Об’єкт – вектор з N ознаками $X = (x_1, x_2, \ldots, x_n)$ у просторі R^n. При навчанні алгоритм повинен побудувати функцію $F(X) = Y$, яка приймає аргумент X – об’єкт з простору R^n і видає мітку класу Y . Головна мета SVM як класифікатора – знайти рівняння роздільної гіперплощини

$$w_1x_1 + w_2x_2 + \ldots + w_nx_n + w_0 = 0$$

в просторі R^n, яка розділила б два класи якимось оптимальним чином.

Загальні відомості про алгоритм

Для простоти розглянемо вибірку об'єктів з двома ознаками x1 та x2 які належать до двох різних класів. Ілюстрацію наведено на рисунку 3.1. Загальний вид перетворення F об'єкта X на мітку класуY :

$$F(X) = \text{sign}(w^T X - b), \text{ де } w = (w_1, w_2, \ldots, w_n),\ b = -w_0.$$

Після налаштування ваг алгоритму w та b (навчання), всі об'єкти, що потрапляють по одну сторону від побудованої гіперплощини, передбачатимуться як перший клас, а об'єкти, що потрапляють по інший бік – другий клас.

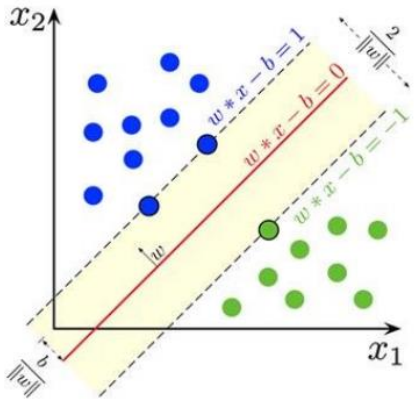


Рис. 2 Робота алгоритму SVM

Сині кружки – об'єкти першого класу; зелені – об'єкти другого класу; кружки з чорним контуром – опорні об'єкти; пряма $wx - b = 0$ оптимально розділяє два класи; прямі $wx - b = \pm 1$, які проходять через опорні вектори x+ та x− визначають ширину розділяючої полоси, як проекцію вектора ($x_+ - x_-$) на вектор нормалі до розділяючої прямої (гіперплощини) w. Усередині функції $sign()$ стоїть лінійна комбінація ознак об'єкта з вагами алгоритму, саме тому SVM відноситься до лінійних алгоритмів. Розділяючу гіперплощину можна побудувати різними способами, але в SVM ваги w і b налаштовуються таким чином, щоб об'єкти класів лежали якнайдалі від роздільної гіперплощини. Іншими словами, алгоритм максимізує зазор (англ. margin) між гіперплощиною та об'єктами класів, які розташовані найближче до неї – опорні вектори x_+ та x_-.

Плюси, мінуси та модифікації

Плюси:

- ✓ добре працює із простором ознак великого розміру;
- ✓ добре працює з даними невеликого обсягу;
- ✓ алгоритм максимізує смугу, що розділяє, яка, як подушка безпеки, дозволяє зменшити кількість помилок класифікації;

- ✓ оскільки алгоритм зводиться до розв'язання задачі квадратичного програмування у опуклій області, то таке завдання завжди має єдине рішення (роздільна гіперплощина з певними гіперпараметрами алгоритму завжди одна).

Мінуси:

- ❖ довгий час навчання (для великих наборів даних);
- ❖ нестійкість до шуму: викиди у навчальних даних стають опорними об'єктами-порушниками і безпосередньо впливають на побудову роздільної гіперплощини;
- ❖ не описані загальні методи побудови ядер і спрямовуючих просторів, що найбільш підходять для конкретного завдання у разі лінійної нероздільності класів. Підбирати корисні перетворення даних – мистецтво.

Модифікації алгоритму:

- ➢ Метод релевантних векторів (Revance Vector Machine, RVM)
- ➢ 1-norm SVM (LASSO SVM)
- ➢ Doubly Regularized SVM (ElasticNet SVM)
- ➢ Support Features Machine (SFM)
- ➢ Relevance Features Machine (RFM).

Зменшення розмірності методом головних компонент (PCA)

В аналізі даних, як і в будь-якому іншому аналізі, часом буває незайвим створити спрощену модель, що максимально точно описує реальний стан справ. Часто буває так, що ознаки досить сильно залежать одна від одної та їх одночасна наявність є надмірною. Якщо одна ознака строго залежить від іншої, то знаючи одну, ми завжди знаємо й іншу. Але набагато частіше буває так, що ознаки залежать одна від одної не так строго і (що важливо!) не так очевидно. Але як з'ясувати, який саме набір параметрів добре описує наш набір даних, але при цьому має невелику надмірність? Іншими словами, як зменшити розмірність простору, в якому "живуть" дані, втративши при цьому мінімум інформації?

Способи вирішення цього завдання називаються методами зменшення розмірності (dimensionality reduction). Метод головних компонент (principal components analysis, PCA) – один з них. Мета PCA – вилучення з цих даних потрібної інформації. Що є інформацією, залежить від суті завдання, що розв'язується. Дані можуть містити потрібну нам інформацію, вони можуть бути надлишковими. Однак, у деяких випадках, інформації в даних може не бути зовсім. Розмірність даних — кількість зразків та змінних -– має велике значення для успішного видобутку інформації. Зайвих даних немає — краще, коли їх багато, ніж мало.

Дані завжди (або майже завжди) містять у собі небажану складову, яка називається шумом. Природа цього шуму може бути різною, але, у багатьох випадках, шум – це та частина даних, яка не містить інформації, що шукається. Що вважати шумом, а що інформацією, завжди вирішується з урахуванням поставлених цілей та методів, які використовуються для її досягнення.

Шум і надмірність у даних обов'язково виявляють себе через кореляційні зв'язки між змінними. Похибки даних можуть призвести до появи не систематичних, а випадкових зв'язків між змінними. Поняття ефективного рангу та прихованих, латентних змінних, кількість яких дорівнює цьому рангу, є найважливішим поняттям у PCA.

Розглянемо простіший випадок, коли об'єкти описуються лише двома ознаками: x1 та x2. Такі дані легко зобразити на площині (див. рисунок 3).

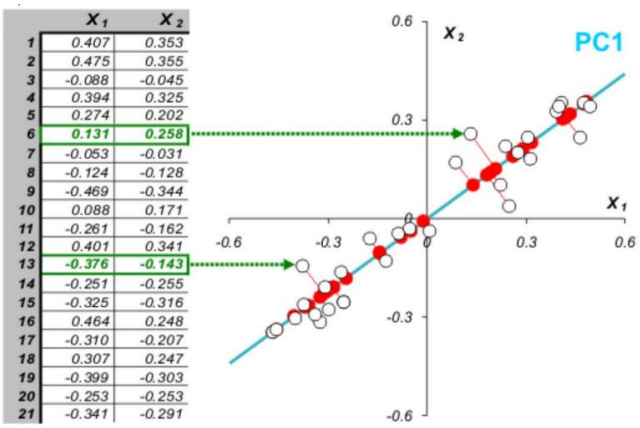


Рис. 3

Кожному рядку вихідної таблиці (тобто зразку) відповідає точка на площині з відповідними координатами. Вони позначені порожніми кружками. Проведемо через них пряму, так, щоб уздовж неї відбувалася максимальна зміна даних. На рисунку ця пряма виділена блакитним кольором; вона називається першою головною компонентою PC_1. Потім проектуємо всі вихідні точки на цю вісь (червоні кружки). Якщо дані описані не повністю (шум великий), то вибирається ще один напрямок (PC_2) – перпендикулярне до першого, так щоб описати зміну, що залишилася в даних і т.д.

Таким чином, знаючи залежності та їх силу, ми можемо виразити кілька ознак через одну, злити докупи, так би мовити, і працювати вже з більш простою моделлю. Звичайно, уникнути втрат інформації, швидше за все, не вдасться, але мінімізувати її нам допоможе якраз метод PCA.

Отже, даний метод апроксимує n-розмірну хмару спостережень до еліпсоїда (теж n-вимірного), півосі якого і будуть майбутніми головними компонентами. І при проекції даних на такі осі (зниженні розмірності) зберігається найбільше інформації.

Зменшення розмірності даних

Метод основних компонентів застосовується до даних, записаним у вигляді матриці X -– прямокутної таблиці чисел розмірністю I рядків і J стовпців. Традиційно рядки цієї матриці називаються об'єктами. Вони нумеруються індексом i = 1, . . . , I. Стовпці називаються ознаками і вони нумеруються індексом j = 1, . . . , J. У методі головних компонент використовуються нові, формальні змінні

t_a (a = 1, . . . , A), що є лінійною комбінацією вихідних змінних x_j (j = 1, . . . , J):

$$t_a = p_{a1}x_1 + p_{a2}x_2 + \cdots + p_{aJ}x_J$$

За допомогою цих нових змінних матриця X розкладається у добуток двох матриць T та P:

$$X = TP^T + E = \sum_{a=1}^{A} t_a p_a^T + E$$

Матриця T називається матрицею рахунків (scores). Її розмірність (I × A). Матриця P називається матрицею навантажень (loadings). Її розмірність (J × A). E – матриця залишків, розмірністю (I × J).

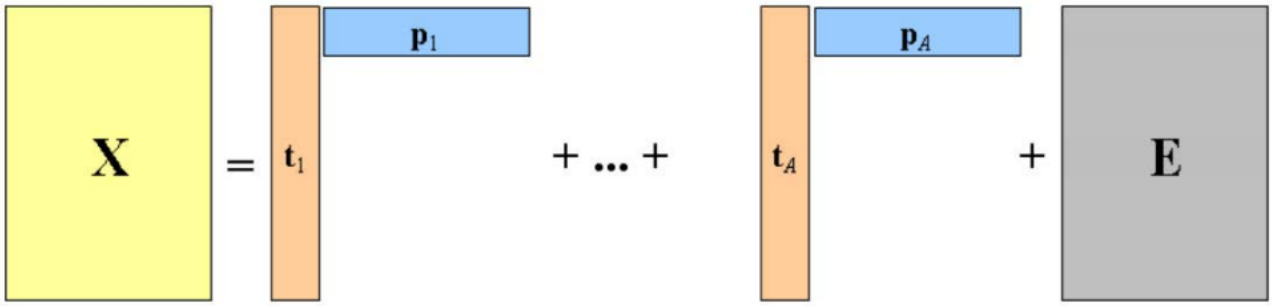


Рис. 4

Нові змінні ta називаються головними компонентами (Principal Components), тому сам метод називається методом головних компонент (PCA). Число стовпців t_a в матриці T, і ра в матриці P дорівнює A, яке називається числом головних компонент (PC). Ця величина свідомо менша від числа змінних J і числа зразків I.

Важливою властивістю PCA є ортогональність (незалежність) основних компонент. Тому матриця рахунків T не перебудовується зі збільшенням числа компонент, а до неї просто додається ще один стовпець – відповідний новому напрямку. Теж відбувається і з матрицею навантажень P.

Статистичні основи

Для опису випадкової величини використовуються моменти. Потрібні нам – математичне очікування та дисперсія. Можна сміливо сказати, що математичне

очікування це “центр тяжіння” величини, а дисперсія – її “розміри” (розкид). Сам процес проектування на вектор ніяк не впливає на значення середніх, тому що для мінімізації втрат інформації наш вектор має проходити через центр нашої вибірки. Тому зазвичай проводять центрування вибірки -– лінійно зміщуючи її так, щоб середні значення ознак дорівнювали 0. Оператор, зворотний зсуву дорівнюватиме вектору початкових середніх значень — він знадобиться відновлення вибірки у вихідній розмірності. При цьому, дисперсія залежить від порядків значень випадкової величини, тобто, є чутливою до масштабування.

Для опису форми випадкового вектора необхідна матриця коваріації. У коваріаційній матриці по діагоналі будуть дисперсії ознак (оскільки i = j), а в інших комірках — коваріації відповідних пар ознак. А в силу симетричності коваріації матриця теж буде симетрична. Коваріаційна матриця є узагальненням дисперсії на випадок багатовимірних випадкових величин вона так само описує форму (розкид) випадкової величини, як і дисперсія, яка для одновимірної випадкової величини має вигляд матриці розміру 1×1, в якій її єдиний член заданий формулою $Cov(x,x) = Var(x)$. Коваріаційна матриця описує форму нашої випадкової величини (ознаки). Узагальнення дисперсії на вищі розмірності – матриця коваріації, і ці два поняття еквівалентні. При проекції на вектор максимізується дисперсія проекції, при проекції на простір великих порядків – вся її коваріаційна матриця.

Власні значення та власні вектори

Тепер треба знайти такі вектори, при якому б максимізувався розкид (дисперсія) проекції вибірки на них. Слід зазначити, що узагальнення дисперсії на вищі розмірності – матриця коваріацій, і ці два поняття еквівалентні. При проекції на вектор максимізується дисперсія проекції, при проекції простору великих порядків – вся її коваріаційна матриця.

Отже, візьмемо одиничний вектор на який проектуватимемо наш випадковий вектор X. Тоді проекція на нього дорівнюватиме v^TX (v^T – транспонування вектора v). Дисперсія проекції на вектор відповідно дорівнює $Var(v^TX)$. Загалом у векторній формі (для центрованих величин) дисперсія виражається так:

$$Var(X) = \Sigma = E(X \cdot X^T)$$

Відповідно, дисперсія проекції:

$$Var(X^*) = \Sigma^* = E(X^* \cdot X^{*T}) = E\left((v^T X) \cdot (V^T X)^T\right) = E\left((v^T X \cdot X^T v\right) = v^T E\left(X \cdot X^T\right) v = v^T \Sigma v$$

Таким чином, що дисперсія максимізується за умови максимального значення $v^T \Sigma v$. Далі зручно використати співвідношення Релея, які для коваріаційних матриць M мають спеціальний випадок:

$$R(M, x) = \frac{X^T M X}{X^T X} = \lambda \frac{X^T X}{X^T X} \lambda$$

і відповідно маємо

$$MX = \lambda X$$

Остання формула є розкладанням матриці на власні вектори та значення: X є власним вектором, а λ – власним значенням. Кількість власних векторів та значень дорівнюють розміру матриці (і значення можуть повторюватися).

Таким чином, приходимо до висновку, що напрямок максимальної дисперсії у проекції завжди збігається з власним вектором, що має максимальне власне значення, яке дорівнює величині цієї дисперсії. Це справедливо також для проекцій на більшу кількість вимірювань – дисперсія (коваріаційна матриця) проекції на m-вимірний простір буде максимальною у напрямку m власних векторів, що мають максимальні власні значення.

Розраховуємо кількість втраченої інформації використовуючи власні значення матриці коваріацій при зменшенні розмірності з n ознак до двох:

$$\Delta I = 1 - \frac{\lambda_1 + \lambda_2}{\sum_{i=1}^{n} \lambda_i} \times 100\%$$

Зменшення розмірності методом сингулярного розкладання (SVD)

Сингулярним розкладанням будь-якої матриці X розміру $n \times d$ є її подання у вигляді добутку трьох матриць: U, Σ, V:

$$X = U \Sigma V^T$$

є обмеження на U, Σ, V:

- U має розміри $n \times n$, Σ має розмір $n \times d$, а V має розмір $d \times d$.
- U і V – ортогональні матриці. Тобто $U^T U = I$ і $V^T V = I$.
- Σ є діагональною матрицею: $\Sigma = diag(\sigma 1, \ldots, \sigma d)$ – сингулярні значення матриці X. Тобто всі елементи в Σ дорівнюють нулю, якщо вони не лежать на діагоналі. Крім того, діагональні елементи в Σ розташовані від найбільшого до найменшого.

Стовбці матриці U – ліві сингулярні вектори. Стовбці матриці V – праві сингулярні вектори.

Встановимо, як значення $\sigma 1, \dots, \sigma d$ діагональної матриці залежать від даних

Знайдемо матрицю коваріації

$$S = \frac{X^T X}{n} = \frac{1}{n} V \Sigma U^T U \Sigma V^T = [U^T U = I] = \frac{1}{n} V \Sigma^2 V^T$$

Таким чином:

- Стовбці $v1, \dots, vd$ матриці V являють собою власний базис матриці коваріацій S;
- $\sigma_1^2/n, \dots, \sigma_d^2/n$ – власні значення матриці коваріацій S;
- Вектори $v1, \dots, vd$ – головні (основні) компоненти (principal components) для матриці даних.

Для будь-якого вектору $\tilde{x}_i = Xv_i$ маємо

$$Var(\tilde{x}_i) = Var(Xv_i) = \frac{\sigma_i^2}{n}, \qquad \tilde{x}_i = Xv_i = \sigma_i^2 u_i$$

Властивість матриці Σ:

$$\sigma_1^2 > \sigma_2^2 > \dots > \sigma_d^2$$

Отже:

- ✓ Перша головна компонента u_1 характеризується тією властивістю, що $\tilde{x}_i$ має максимальну дисперсію серед всіх нормованих лінійних комбінацій стовбців матриці X;
- ✓ $\tilde{x}_d$ має мінімальну дисперсію.

Одна дуже корисна властивість SVD полягає в тому, що він завжди знаходить оптимальний набір факторів, які найкраще передбачають оцінки, відповідно до стандартної міри подібності до матриць (нормою Фробеніуса). Тобто якщо ми використовуємо SVD для пошуку факторів матриці, це найкращі фактори, які можна знайти. Ця властивість оптимальності означає, що нам не потрібно ставити питання, чи може інший набір чисел краще передбачати результати.

Кластеризація методом k-середніх (k-Means)

Алгоритми кластеризації дозволяють розділити об’єкти у випадку, коли класи заздалегідь не зазначені, а кластери мають бути сформовані за схожістю елементів. Тобто відбувається реалізація навчання без учителя.

Відмінний приклад кластеризації — маркери на картах в інтернеті. Коли ви шукаєте всі ресторани азіатської кухні у великому місті, машині доводиться

групувати їх у кружечки з цифрою, інакше браузер зависне в потугах намалювати мільйон маркерів.

Більш складні приклади кластеризації можна згадати у програмах iPhoto або Google Photos, які знаходять обличчя людей на фотографіях та групують їх у альбоми. Програма не знає як звуть ваших друзів, але може відрізнити їх за характерними рисами обличчя.

Кластеризація сьогодні використовують для:

- Сегментація ринку (типів покупців)
- Об’єднання близьких точок на карті
- Стиснення зображень
- Аналіз та розмітки нових даних.
- Детектори аномальної поведінки

Популярні алгоритми: Метод K-середніх (K-means), Mean-Shift, DBSCAN.

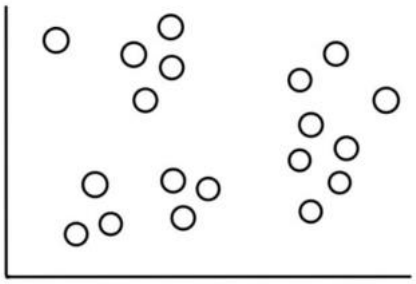
Метод k-середніх (k-means Method)

Даний метод належить до алгоритмів неієрахічного поділу (Partitioning algorithms), які здійснюють декомпозицію набору даних, що складається з n спостережень, на k груп (кластерів) із заздалегідь невідомими параметрами. При цьому виконується пошук центроїдів – максимально віддалених один від одного центрів згущень точок з мінімальним розкидом усередині кожного кластера.

Покрокова реалізація методу

Постановка задачі та попередня обробка даних.

Нехай існує набір даних з n = 19 значень, які виглядають приблизно так:



Тепер припустимо, що ми знаємо, що ці дані розбиваються на три, відносно очевидні категорії і виглядатиме це так:

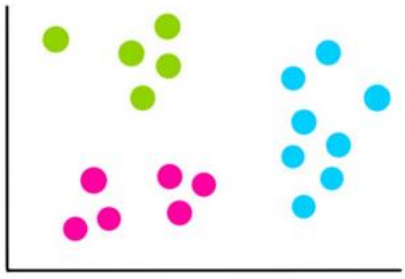


Рис. 5

Завдання – використовувати алгоритм кластеризації k-means, щоб зробити цю категоризацію.

- Крок 1: Вибираємо кількість кластерів, k.

Число кластерів, які ми хочемо розпізнати, це і є k у k-means. У нашому випадку, тому що ми припустили, що всього 3 кластери, k = 3.

- Крок 2: Вибираємо до випадкових значень

Процес виявлення кластерів ми починаємо з вибору трьох випадково обраних значень (не обов'язково, щоб вони були нашими даними). Ці точки зараз працюватимуть як центроїди або центри кластерів, які ми збираємося згрупувати:

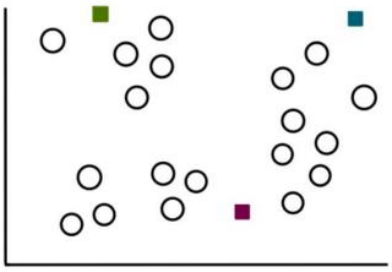


Рис. 6

- Крок 3: Створення кластерів k

Щоб створити кластери, ми почнемо вимірювати відстань між кожним значенням наших даних до кожного з трьох центроїдів, а потім додаємо його до найближчого кластера.

Для значення, взятого як приклад, відстані виглядатимуть приблизно так:

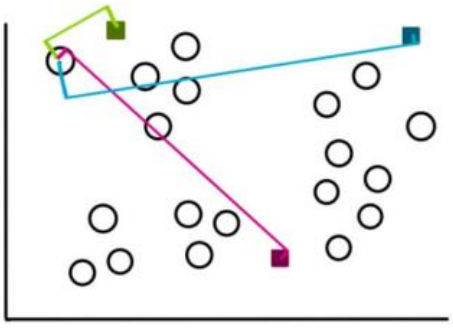


Рис. 7

Для розрахунку відстані між об'єктами буде використовувати стандартне визначення декартової відстані:

$$d = \sqrt{\sum_{i=1}^{m}(x_i - c_{ij})^2},$$

де x_i – координата в m-вимірному просторі (значення кожної ознаки); c_{ij} – координата кожного центроїда. У 2-мірному просторі для знаходження відстані між двома точками маємо:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Використовуючи цю формулу, ми повторюємо процес зі значеннями, що залишилися, після цього кластери будуть виглядати наступним чином:

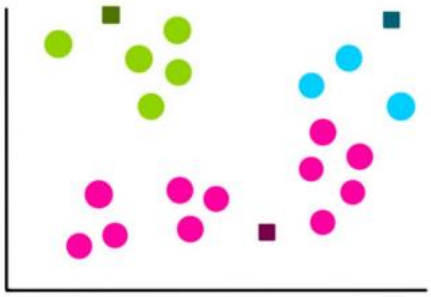


Рис. 8

- Крок 4: Обчислюємо новий центроїд кожного кластера

Тепер, коли ми маємо всі три кластери, ми знаходимо нові центроїди для кожного з них за формулою:

$$(c_{1j}, c_{2j}, \ldots, c_{mj}) = \left(\frac{1}{n_j}\sum_{i=1}^{n_j} x_{1i}, \frac{1}{n_j}\sum_{i=1}^{n_j} x_{2i}, \ldots, \frac{1}{n_j}\sum_{i=1}^{n_j} x_{mi}\right)$$

c_{kj} – k-та координата j-того кластера;

x_{ki} – k-та координата i-того об’єкта, що віднесений до jтого кластера;

n_j – кількість об’єктів у j-тому кластері.

Так, нові центроїди матимуть вигляд:

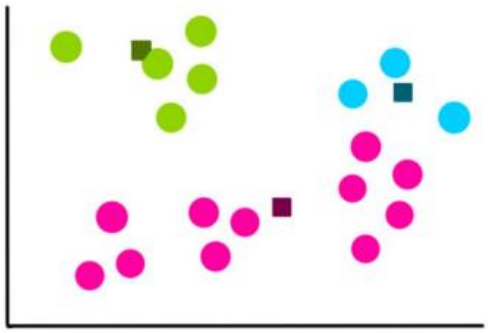


Рис. 9

Крок 5: Знову віднесемо об’єкти до кожного з центроїдів

Деякі об’єкти віднесуться до іншого центроїду (кластеру). Розраховуємо суму квадратів внутрішньокластерних відстаней до центру кластера (within-cluster sum of squares, WCSS):

$$Q = \sum_{j=1}^{k} \sum_{i=1}^{n} \min \left(||x_i^j - c_j|| \right)^2$$

- Крок 6: повторюємо кроки 4-5

Кроки 4 і 5 повторюються до тих пір, поки алгоритм не стабілізується, тобто до тих пір, поки об’єкти не перестануть переходити від одного центроїду (кластера) до іншого. Говорячи формально, мета алгоритму – мінімізувати функція втрат Q.

Давайте припустимо, що наступні 4 ітерації виглядатимуть так:

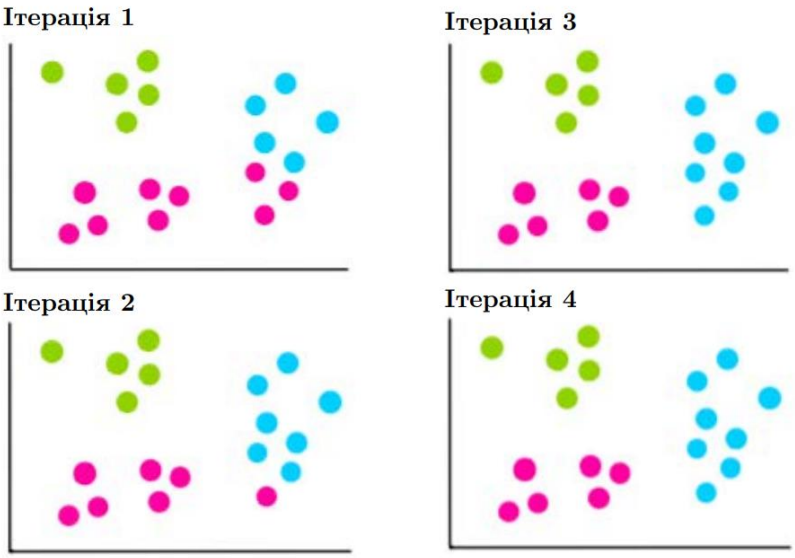


Рис. 10

На двох останніх ітераціях бачимо, що кластери не змінюються. Це означає, що алгоритм зійшовся і ми зупиняємо процес. Потім ми вибираємо кластери із

найменшим WCSS. Це будуть кластери на останніх 2 ітераціях, тому вони стають нашими фінальними кластерами.

Вибір кількості кластерів

Один із способів вибору кількості кластерів є експертний метод – вибір кількості кластерів залежатиме від знання про предметну область (domain knowledge).

При кластеризації методом k-середніх кількість кластерів найчастіше оцінюють за допомогою «методу ліктя» (elbow method).

Він передбачає багаторазове циклічне виконання алгоритму зі збільшенням кількості кластерів, що вибираються.

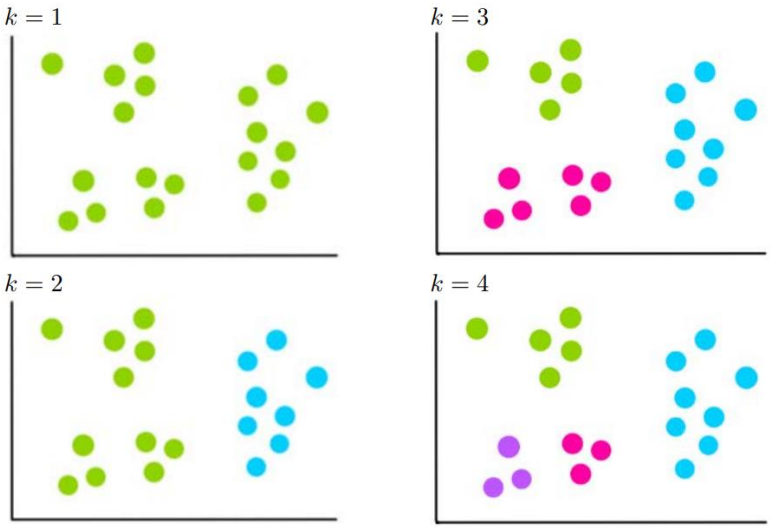


Рис. 11

Для кожного вибору отримаємо мінімальне значення суму квадратів внутрішньокластерних відстаней Q. За результатами досліджень будуємо графік залежності Q(j).

На рисунку 12 наведено типову залежність Q(j). Як бачимо, після того, як кількість кластерів досягає трьох, сума квадратів внутрішньокластерних відстаней перестає суттєво зменшуватися.

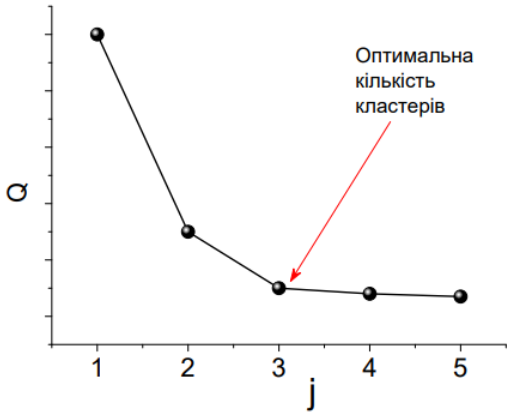


Рис. 12

Кластеризація методом DBscan

Маючи справу з просторовими кластерами різної щільності, розміру та форми, може бути складно виявити групу точок. Завдання може бути ще складнішим, якщо дані містять шум і викиди. Для роботи з великими просторовими базами даних був запропонований метод DBSCAN (Density-based spatial clustering of

applications with noise, густинний алгоритм просторової кластеризації з присутністю шуму), який, як випливає з назви, оперує щільністю даних.

На вхід він просить вже знайому матрицю близькості та два параметри – радіус epsilon-околиці та кількість сусідів. Так одразу і не зрозумієш, як їх вибрати, причому тут щільність, і чому саме DBSCAN добре розправляється з шумовими даними. Без цього складно визначити межі його застосування. Можна виділити три основні причини використання алгоритму:

1. Вимагає мінімальних знань предметної галузі.
2. Він може виявити кластери довільної форми.
3. Ефективний для великих баз даних, тобто коли розмір вибірки перевищує кілька тисяч.

Формальний підхід

Уведемо кілька визначень. Нехай задана деяка симетрична функція відстані $\rho(x, y)$ та константи ϵ і m. Тоді:

1. Назвемо область E(x), для якої ∀y : ρ(x, y) ≤ ϵ, ϵ-околиця об'єкта x.

2. Кореневим об'єктом (core point) ступеня m називається об'єкт, ϵ-околиця якого містить щонайменше m об'єктів: |E(x)| ≥ m.

3. Об'єкт p безпосередньо щільно досягається з об'єкта q, якщо $p \in E(q)$ і q – кореневий об'єкт.

4. Об'єкт p щільно-досяжний з об'єкта q, якщо ∃p_1, p_2 , …, p_n, p_1 = q, p_n = p, такі що ∀i ∈ 1 . . . n − 1 : p_{i+1} безпосередньо щільнодосяжний з p_i.

Оберемо якийсь кореневий об'єкт p з датасету, позначимо його і помістимо всіх його безпосередньо щільно-досяжних сусідів у список обходу. Тепер для кожного об'єкту q зі списку: позначимо його, і якщо він теж є кореневим, додамо всіх його сусідів до списку обходу. Тривіально доводиться, що кластери помічених точок, сформовані в ході цього алгоритму максимальні (тобто їх не можна розширити ще однією точкою, щоб задовольнялися умови) і зв'язні в сенсі досяжності. Звідси випливає, що якщо ми обійшли не всі точки, можна перезапустити обхід з якогось іншого кореневого об'єкта, і новий кластер не поглине попередній.

Налаштування параметрів

Існують варіанти DBSCAN, які здатні пом'якшити цю проблему.

Ідея полягає у підстроюванні ϵ у різних галузях по ходу роботи алгоритму. На жаль, зростає кількість параметрів алгоритму.

Існують евристики для вибору m та ϵ. Найчастіше застосовується такий метод та його варіації:

1. Обираємо m. Зазвичай використовуються значення від 3 до 9, чим неоднорідніший очікується датасет, і чим більший рівень шуму, тим більшим слід взяти m.
2. Обчислюємо середню відстань по m найближчим сусідам кожної точки. Тобто, якщо m = 3, потрібно вибрати трьох найближчих сусідів, скласти відстані й поділити втричі.
3. Сортуємо отримані значення за зростанням та виводимо на екран.
4. Отримуємо зростаючу залежність. Обираємо є десь у смузі, де відбувається найсильніший перегин. Чим більше є, тим більшими вийдуть кластери, і тим менше їх буде.

У будь-якому випадку, головні недоліки DBSCAN – нездатність з'єднувати кластери через прорізи, і, навпаки, здатність пов'язувати різні кластери через щільно населені перемички. Частково тому зі збільшенням розмірності даних n підступний "удар у спину" завдає «прокляття» розмірності: що більше n, то більше місць, де можуть випадково виникнути отвори чи мости. Адекватна кількість точок даних N зростає експоненційно зі збільшенням n.

Підсумок

DBSCAN слід використовувати, коли:

- Датасет є в міру великий (N approx106). Навіть N ≈ 107 −108 якщо під рукою оптимізована та розпаралена реалізація.
- Наперед відома функція близькості, симетрична, бажано, не дуже складна. KD-Tree оптимізація часто працює тільки з евклідовою відстанню.
- Очікувані кластери складної форми: вкладені та аномальні кластери, згустки даних малої розмірності.
- Щільність меж між згустками менше щільності найменш щільного кластера. Краще якщо кластери зовсім відокремлені один від одного.
- Складність елементів датасета значення не має. Однак їх має бути достатньо, щоб не виникало сильних розривів у густині.
- Кількість елементів у кластері може змінюватись як завгодно.
- Кількість викидів значення не має (в розумних межах), якщо вони розсіяні у великому об'ємі.
- Кількість кластерів значення немає.

Сфери застосування

DBSCAN має історію успішних застосувань. Наприклад, можна відзначити його використання у завданнях:

- ✓ Виявлення соціальних гуртків
- ✓ Сегментування зображень
- ✓ Моделювання поведінки користувачів веб-сайтів
- ✓ Попередня обробка в задачі прогнозування погоди

Пошук асоціативних правил з використанням методу Apriori

Асоціація шукає закономірності у потоці замовлень

Навчання асоціативним правилам або пошук асоціативних правил — це метод навчання машин на базі правил виявлення зв'язків, що цікавлять нас, між змінними у великій базі даних.

Метод пропонується для встановлення сильних правил, виявлених у базі даних за допомогою деяких заходів цікавості. Цей заснований на правилах підхід також генерує нові правила в міру аналізу додаткових даних. Кінцевою метою, виходячи з досить великого набору даних, допомогти машині імітувати виділення людських ознак і створити можливість знаходження абстрактних асоціацій з нових некласифікованих даних.

Сьогодні використовують для:

- Прогноз акцій та розпродажів
- Аналіз товарів, що купуються разом
- Розміщення товарів на полицях
- Аналіз патернів поведінки на веб-сайтах

Популярні алгоритми: Apriori, FP-growth.

Алгоритм APriory – пошук в ширину

Кроки в Apriory

1) На першій ітерації алгоритму кожен предмет приймається як кандидат набору з одного предмета. Алгоритм буде рахувати випадки появи кожного елемента.

2) Нехай буде якась мінімальна підтримка, min_sup. Визначається набір 1-елементних наборів предметів, поява яких задовольняє мінімальну суму. Тільки тих кандидатів, для яких підтримка більша або дорівнює min_sup, приймають на наступну ітерацію, а інших обрізають.

3) Далі виявляються часті набори предметів. Для цього на етапі об’єднання набір із двох елементів формується шляхом формування групи з 2 шляхом комбінування елементів між собою.

4) Кандидати з 2 елементів обрізаються з використанням порогового значення min-sup. Тепер у таблиці будуть дво-елементні набори з мінімальною сумою.

5) Наступна ітерація сформує три-елементні набори, використовуючи кроки об’єднання та обрізки. Ця ітерація буде слідувати властивості антимонотонності. Якщо всі підмножини з 2 елементів є частими, тоді надмножина буде частою, інакше вона буде обрізана.

6) Наступним кроком буде створення набору 4-елементів, з'єднавши 3-набір елементів із собою та обрізання, якщо його підмножина не відповідає критеріям min_sup. Алгоритм зупиняється, коли досягається найчастіший набір елементів.



Рис. 13



Рис.14

Приклад Apriory: поріг підтримки = 50%, впевненість = 60%

Транзакція	Перелік предметів
T1	I1, I2, I3
T2	I2, I3, I4
T3	I4, I5
T4	I1, I2, I4
T5	I1, I2, I3, I5
T6	I1, I2, I3, I4

Набір	Підтримка
I1, I2	4
I1, I3	3
I2, I3	4
I2, I4	3

Поріг підтримки = 50%
⇒ 0,5 * 6 = 3
⇒ min_sup = 3

Набір	Підтримка
I1, I2, I3	3
I1, I2, I4	1
I1, I3, I4	1
I2, I3, I4	2

Лише набір елементів {I1, I2, I3} відповідає min_sup. Він є частим.

Рис.15

Приклад Apriory: поріг підтримки = 50%, впевненість = 60%

Транзакція	Перелік предметів
T1	I1, I2, I3
T2	I2, I3, I4
T3	I4, I5
T4	I1, I2, I4
T5	I1, I2, I3, I5
T6	I1, I2, I3, I4

Елемент	Підтримка
I1	4
I2	5
I3	4
I4	4

Набір	Підтримка
I1, I2	4
I1, I3	3
I2, I3	4
I2, I4	3

Набір	Підтримка
I1, I2, I3	3

Сформуємо правила асоціації: для трьох частих наборів предметів, виявлених вище:

- {I1, I2} => {I3}
- {I1, I3} => {I2}
- {I2, I3} => {I1}
- {I1} => {I2, I3}
- {I2} => {I1, I3}
- {I3} => {I1, I2}

Рис. 16

Елемент	Підтримка
I1	4
I2	5
I3	4
I4	4

Набір	Підтримка
I1, I2	4
I1, I3	3
I2, I3	4
I2, I4	3

Набір	Підтримка
I1, I2, I3	3

впевненість = 60%

Розрахуємо впевненість для шести правил:

- {I1, I2} => {I3}

Впевненість = підтримка {I1, I2, I3} / підтримка {I1, I2} = (3/4) * 100 = 75%

- {I1, I3} => {I2}

Впевненість = підтримка {I1, I2, I3} / підтримка {I1, I3} = (3/3) * 100 = 100%

- {I2, I3} => {I1}

Впевненість = підтримка {I1, I2, I3} / підтримка {I2, I3} = (3/4) * 100 = 75%

- {I1} => {I2, I3}

Впевненість = підтримка {I1, I2, I3} / підтримка {I1} = (3/4) * 100 = 75%

- {I2} => {I1, I3}

Впевненість = підтримка {I1, I2, I3} / підтримка {I2} = (3/5) * 100 = 60%

- {I3} => {I1, I2}

Впевненість = підтримка {I1, I2, I3} / підтримка {I3} = (3/4) * 100 = 75%

Рис. 17

Програмна реалізація

Завдання були виконані у Jupiter Notebook.

Метод дерева прийняття рішень (decision tree)

Лабораторна робота 1

Task 1
Постановка задачі

1. Розділити всю вибірку на навчальну та тестову
2. Побудувати алгоритм навчання на навчальній вибірці з використанням вбудованого алгоритму DecisionTreeClassifier() з бібліотеки sklearn
3. Перевірити результати роботи на тестовій вибірці.
4. Вивести на екран дерево.
5. Оформити результати у вигляді звіту.

Розглянемо перший датасет

- Імпортуємо необхідні бібліотеки

In [1]:

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
```

In [2]:

```
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

# Імпорт модуля метрик scikit-learn для розрахунку точності
from sklearn import metrics

from sklearn.tree import export_graphviz
from io import StringIO

from IPython.display import Image
import graphviz
import pydotplus
```

In [3]:

```
!pip install graphviz
!pip install pydotplus
Requirement already satisfied: graphviz in c:\users\admin\anaconda3\lib\site-
packages (0.20.1)
Requirement already satisfied: pydotplus in c:\users\admin\anaconda3\lib\site-
packages (2.0.2)
Requirement already satisfied: pyparsing>=2.0.1 in
c:\users\admin\anaconda3\lib\site-packages (from pydotplus) (3.0.9)
```

In [4]:

```
#conda install graphviz
```

- Будемо розглядати датасет **creditcard.csv**. Виведемо його у формі датафрейму.

In [5]:

```
data = pd.read_csv('creditcard.csv')
```

In [6]:

```
data.head()
```

Out[6]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.2778
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.6386
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.7716

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.0052
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.7982

5 rows × 31 columns

In [7]:

```
#data.info()
```

- Далі визначаємо ознаки та цільову змінну. Ділимо вибірку на навчальну і тестову. Виведемо їх розмірності.

In [8]:

```
X = data.drop(columns='Class', axis=1)
y = data['Class']
```

In [9]:

```
data['Class'].value_counts()
```

Out[9]:

```
0    284315
1       492
Name: Class, dtype: int64
```

In [10]:

```
X_train, X_test, y_train, y_test = \
                train_test_split(X, y, test_size = 0.25, random_state = 1) #
75% training and 25% test
print(X_train.shape, X_test.shape)
print(y_train.shape, y_test.shape)
(213605, 30) (71202, 30)
(213605,) (71202,)
```

- Створюємо об'єкт класифікатора дерева рішень. За замовчуванням у класифікатора стоїть критерій *gini* (default="gini"), тому можна не вказувати його. Максимальна глибина дерева буде 3.
- Створюємо класифікатор дерева прийняття рішень для тренувань.
- Робимо прогноз на тестовому наборі даних.

In [11]:

```
clf = DecisionTreeClassifier(random_state=0, max_depth=3)
clf = clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

- Перевіряємо результати роботи на тестовій вибірці за допомогою точності моделі (як часто класифікатор є правильним):

In [12]:

```
y_pred = clf.predict(X_test)
print("Оцінка тестового набору даних: ", metrics.accuracy_score(y_test, y_pred))
Оцінка тестового набору даних:  0.9991713715906857
```

- Виводимо на екран дерево прийняття рішень:

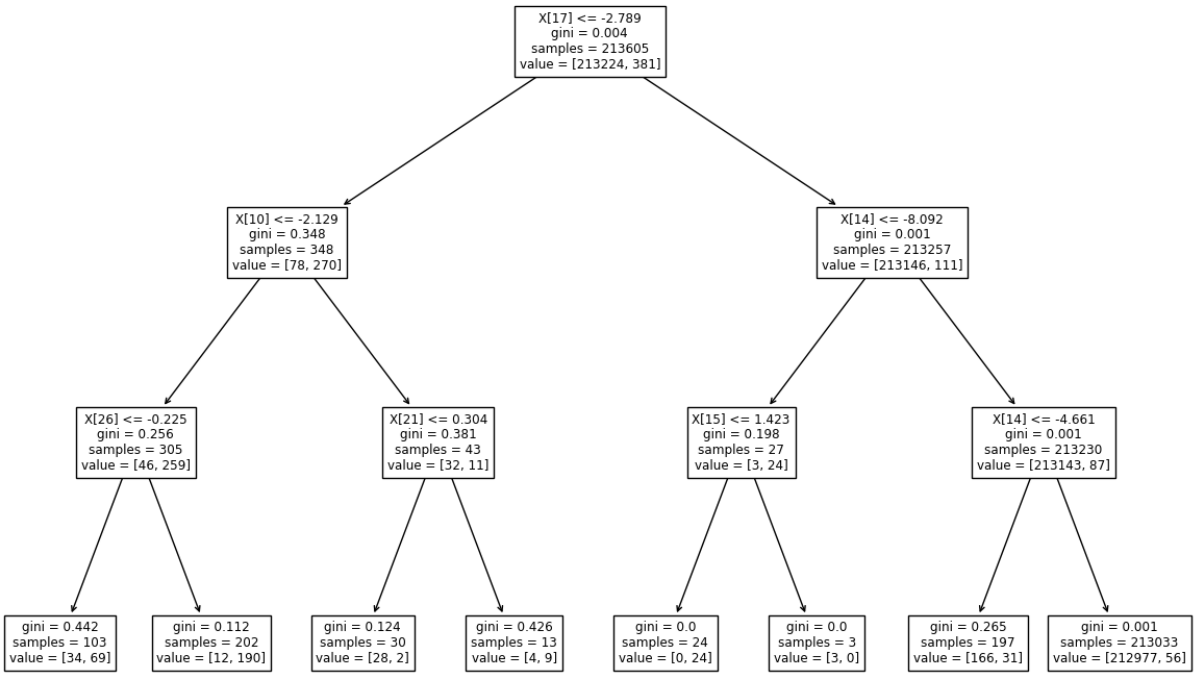
In [13]:

```
plt.figure(figsize=(15,10))
tree.plot_tree(clf.fit(X_train, y_train))
```

Out[13]:

```
[Text(0.5, 0.875, 'X[17] <= -2.789\ngini = 0.004\nsamples = 213605\nvalue =
[213224, 381]'),
 Text(0.25, 0.625, 'X[10] <= -2.129\ngini = 0.348\nsamples = 348\nvalue = [78,
270]'),
 Text(0.125, 0.375, 'X[26] <= -0.225\ngini = 0.256\nsamples = 305\nvalue = [46,
259]'),
 Text(0.0625, 0.125, 'gini = 0.442\nsamples = 103\nvalue = [34, 69]'),
 Text(0.1875, 0.125, 'gini = 0.112\nsamples = 202\nvalue = [12, 190]'),
 Text(0.375, 0.375, 'X[21] <= 0.304\ngini = 0.381\nsamples = 43\nvalue = [32,
11]'),
 Text(0.3125, 0.125, 'gini = 0.124\nsamples = 30\nvalue = [28, 2]'),
 Text(0.4375, 0.125, 'gini = 0.426\nsamples = 13\nvalue = [4, 9]'),
 Text(0.75, 0.625, 'X[14] <= -8.092\ngini = 0.001\nsamples = 213257\nvalue =
[213146, 111]'),
```

```
 Text(0.625, 0.375, 'X[15] <= 1.423\ngini = 0.198\nsamples = 27\nvalue = [3,
24]'),
 Text(0.5625, 0.125, 'gini = 0.0\nsamples = 24\nvalue = [0, 24]'),
 Text(0.6875, 0.125, 'gini = 0.0\nsamples = 3\nvalue = [3, 0]'),
 Text(0.875, 0.375, 'X[14] <= -4.661\ngini = 0.001\nsamples = 213230\nvalue =
[213143, 87]'),
 Text(0.8125, 0.125, 'gini = 0.265\nsamples = 197\nvalue = [166, 31]'),
 Text(0.9375, 0.125, 'gini = 0.001\nsamples = 213033\nvalue = [212977, 56]')]
```



- Візуалізуємо дерево більш детально

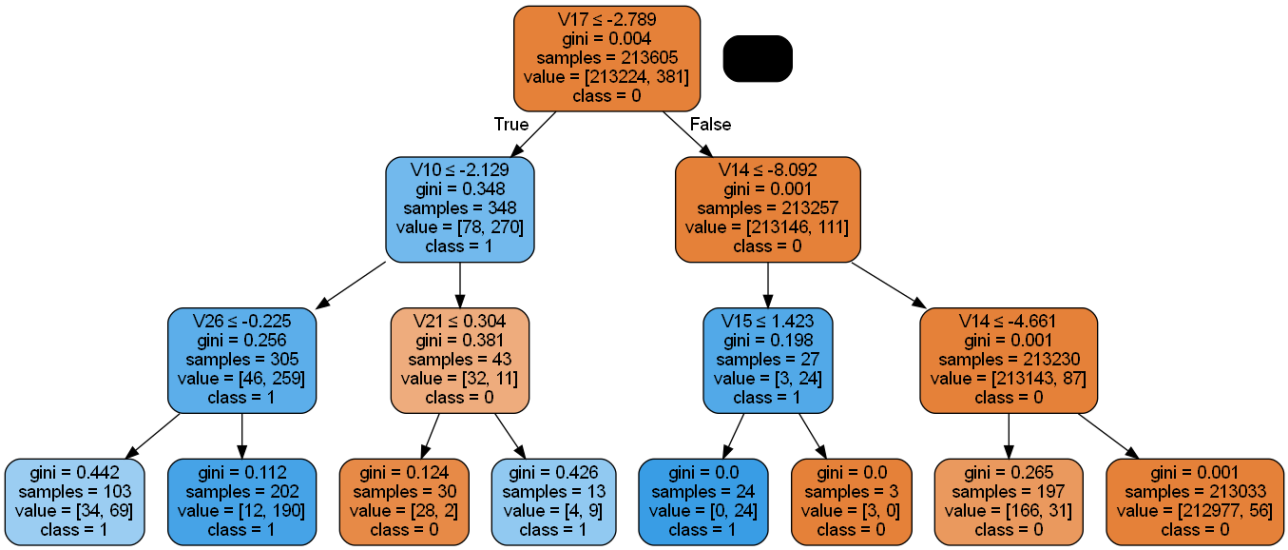
In [14]:

```
feature_cols_credit = [i for i in X.columns]

dot_data_credit = StringIO()
export_graphviz(clf, out_file = dot_data_credit,
                feature_names = feature_cols_credit,
                class_names = ["0", "1"],
                filled = True, rounded = True,
                special_characters = True)

graph_credit = pydotplus.graph_from_dot_data(dot_data_credit.getvalue())
graph_credit.write_png('lab1_credit.png')
Image(graph_credit.create_png())
```

Out[14]:



''' %matplotlib inline feature_cols_credit = [i for i in X.columns] dot_data_credit = tree.export_graphviz(clf, out_file = None, feature_names = feature_cols_credit, class_names = ["0", "1"], filled = True, rounded = True, special_characters = True) graph_credit = graphviz.Source(dot_data_credit) graph_credit.save('lab1_credit.png') graph_credit '''

Розглянемо тепер інший датасет

- Імпортуємо необхідні бібліотеки

In [15]:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text
```

- Будемо розглядати датасет **iris**. Далі визначаємо ознаки та цільову змінну. Виведемо датасет у формі датафрейму

In [16]:

```
iris = load_iris()
```

In [17]:

```
X2, y2 = iris.data, iris.target
```

In [18]:

```
# dfIris = pd.DataFrame(data = np.c_[iris['data'], iris['target']], columns =
iris['feature_names'] + ['target'])
dfIris = pd.DataFrame(data = np.c_[X2, y2], columns = iris['feature_names'] +
['target'])
dfIris
```

Out[18]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2.0
146	6.3	2.5	5.0	1.9	2.0
147	6.5	3.0	5.2	2.0	2.0
148	6.2	3.4	5.4	2.3	2.0
149	5.9	3.0	5.1	1.8	2.0

150 rows × 5 columns

Класи розглянутих об'єктів:

In [19]:

```
print(iris['target_names'])
['setosa' 'versicolor' 'virginica']
```

In [20]:

```
print(dfIris['target'].unique())
[0. 1. 2.]
```

- Ділимо вибірку на навчальну і тестову. Виведемо їх розмірності.

In [21]:

```
X2_train, X2_test, y2_train, y2_test = \
                train_test_split(X2, y2, test_size = 0.25, random_state = 1) #
75% training and 25% test
print(X2_train.shape, X2_test.shape)
print(y2_train.shape, y2_test.shape)
(112, 4) (38, 4)
(112,) (38,)
```

- Створюємо об'єкт класифікатора дерева рішень. За замовчуванням у класифікатора стоїть критерій *gini* (default="gini"), тому можна не вказувати його. Максимальна глибина дерева буде 3.
- Створюємо класифікатор дерева прийняття рішень для тренувань.
- Робимо прогноз на тестовому наборі даних.

In [22]:

```
clf2 = DecisionTreeClassifier(random_state=0, max_depth=3)
clf2 = clf2.fit(X2_train, y2_train)
y2_pred = clf2.predict(X2_test)
```

- Перевіряємо результати роботи на тестовій вибірці за допомогою точності моделі (як часто класифікатор є правильним):

In [23]:

```
y2_pred = clf2.predict(X2_test)
print("Оцінка тестового набору даних: ", metrics.accuracy_score(y2_test,
y2_pred))
Оцінка тестового набору даних:  0.9736842105263158
```

- Виводимо на екран дерево прийняття рішень:

In [24]:

```
#plt.figure(figsize=(10,9))
#tree.plot_tree(clf2.fit(X2_train, y2_train))
#plt.title("Дерево прийняття рішень (decision tree)")
```

In [25]:

```
decision_tree_iris = clf2
tx = export_text(decision_tree_iris, feature_names=iris['feature_names'])
print(tx)
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) >  0.80
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- class: 2
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2
```

In [26]:

```
dot_data = tree.export_graphviz(clf2, out_file = None,
                      feature_names = iris.feature_names,
                      class_names = ["0", "1", "2"],
                      filled = True, rounded = True,
                      special_characters = True)
graph = graphviz.Source(dot_data)
#graph.save('lab1_iris.png')
graph.render("lab1_iris")
graph
```

Out[26]:

petal width (cm) ≤ 0.8 gini = 0.665 samples = 112 value = [37, 34, 41] class = 2 gini = 0.0 samples = 37 value = [37, 0, 0] class = 0 True petal width (cm) ≤ 1.65 gini = 0.496 samples = 75 value = [0, 34, 41] class = 2 False petal length (cm) ≤ 4.95 gini = 0.193 samples = 37 value = [0, 33, 4] class = 1 petal length (cm) ≤ 4.85

gini = 0.051 samples = 38 value = [0, 1, 37] class = 2 gini = 0.0 samples = 32 value = [0, 32, 0] class = 1 gini = 0.32 samples = 5 value = [0, 1, 4] class = 2 gini = 0.375 samples = 4 value = [0, 1, 3] class = 2 gini = 0.0 samples = 34 value = [0, 0, 34] class = 2

- Наступний варіант з назвами класів ірисів.

In [27]:

```
dot_data2 = tree.export_graphviz(clf2, out_file = None,
                     feature_names = iris.feature_names,
                     class_names = iris.target_names,
                     filled = True, rounded = True,
                     special_characters = True)
graph2 = graphviz.Source(dot_data2)
#graph2.save('lab1_iris_classColors.jpg')
graph2.render("lab1_iris_classColors")
graph2
```

Out[27]:

petal width (cm) ≤ 0.8 gini = 0.665 samples = 112 value = [37, 34, 41] class = virginica gini = 0.0 samples = 37 value = [37, 0, 0] class = setosa True petal width (cm) ≤ 1.65 gini = 0.496 samples = 75 value = [0, 34, 41] class = virginica False petal length (cm) ≤ 4.95 gini = 0.193 samples = 37 value = [0, 33, 4] class = versicolor petal length (cm) ≤ 4.85 gini = 0.051 samples = 38 value = [0, 1, 37] class = virginica gini = 0.0 samples = 32 value = [0, 32, 0] class = versicolor gini = 0.32 samples = 5 value = [0, 1, 4] class = virginica gini = 0.375 samples = 4 value = [0, 1, 3] class = virginica gini = 0.0 samples = 34 value = [0, 0, 34] class = virginica
''' dot_data = StringIO() export_graphviz(clf2, out_file = None, feature_names = iris.feature_names, class_names = ["0", "1", "2"], filled = True, rounded = True, special_characters = True) graph = pydotplus.graph_from_dot_data(dot_data.getvalue()) graph.write_png('lab1_iris.png') Image(graph.create_png()) '''

Машинне навчання

Метод класифікації kNN

Лабораторна робота 2

Метод класифікації kNN

Етапи розв'язання

1. Зафіксувати значення параметра k
2. Для кожного об'єкта з тестової вибірки:
   Розрахувати відстань від об'єкта з набором ознак a_i до кожного об'єкта з навчальної вибірки з ознаками f_i за формулою

$$D_j = \sqrt{\sum_{i=1}^{l} (f_i - a_i)^2}$$

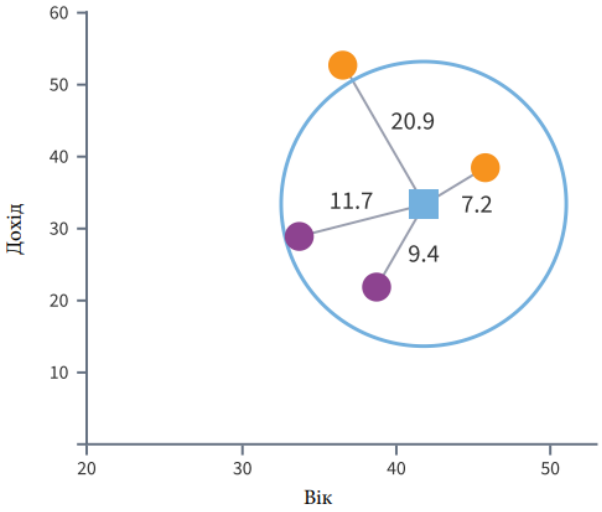
   Розрахувати поеказник близькості

$$Q_j = \sum_{i=1}^{n_j} \frac{1}{D^2(f, a_{ij})}$$

   Визначити до якого класу з навчальної вибірки належить об'єкт
3. Порахувати кількість помилок класифікації шляхом порівняння результатів класифікації з відомими значеннями класів для кожного об'єкта з тестової вибірки
4. Провести оптимізацію параметра k
5. Порівняти результати з kNN з sklearn
6. Оформити результати у вигляді звіту

1.1.5 Чисельний приклад

Розглянемо простий чисельний приклад роботи алгоритму KNN, проілюстрований на рисунку 2.2. Нехай є набір даних про позичальників банку частина з яких допустили прострочення платежу (таблиця). Ознаками є вік та середньомісячний дохід. Мітками класу в полі "Прострочено" будуть "Так" та "Ні".



X_1 – вік	X_2 – дохід	Прострочено
46	40	Ні
36	54	Ні
34	29	Так
38	23	Так

Робота алгоритму KNN

На рисунку помаранчевими кружками представлені об'єкти класу "Ні", а фіолетовими класу "Так". Синім квадратом відображається об'єкт, що класифікується (новий позичальник).

Завдання полягає в тому, щоб виконати класифікацію нового позичальника для якого $a_1 = 42$ і $a_2 = 34$ з метою оцінити можливість прострочення ним платежів.

1. Задамо значення параметра $k = 3$

2. Розрахуємо відстань між вектором ознак об'єкта, що класифікується, і векторами навчальних прикладів за формулою (1.1) та встановимо для кожного прикладу його ранг.

X_1 – вік	X_2 – дохід	Відстань	Ранг	Сусід	Прострочено
46	40	7.2	1	Так	Ні
36	54	20.9	4	Ні	Ні
34	29	9.4	2	Так	Так
38	23	11.7	3	Так	Так

3. Виключимо з розгляду приклад, який при $k = 3$ не є сусідом і розглянемо класи, що залишилися.

X_1 – вік	X_2 – дохід	Відстань	Ранг	Сусід	Прострочено
46	40	7.2	1	Так	Ні
34	29	9.4	2	Так	Так
38	23	11.7	3	Так	Так

Таким чином, з трьох найближчих сусідів (на рисунку розташовані всередині кола) об'єкта, що класифікується, два мають клас "Так", а один – "Ні". Отже, шляхом простого невиваженого голосування визначаємо його клас як "Так".

Для проведення зваженого голосування розрахуємо показник близькості за формулою (1.2):

$$Q_{NO} = \frac{1}{65} \approx 0.015 \qquad Q_{YES} = \frac{1}{98} + \frac{1}{137} \approx 0.018.$$

Шляхом зваженого голосування визначаємо клас об'єкта, що класифікується, як "Так".

З роботи класифікатора робимо висновок, що позичальник із заданими характеристиками може допустити прострочення виплати кредиту.

- Імпортуємо потрібні бібліотеки

In [1]:
```
# import warnings filter
from warnings import simplefilter
# ignore all future warnings
simplefilter(action='ignore', category=FutureWarning)
```

In [2]:
```
import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib import pyplot as plt
```

In [3]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
```

- Розглянемо датасет з ірисами. Створимо датафрейм з нього.

In [4]:
```
iris = load_iris()
X, y = iris.data, iris.target
```

In [5]:
```
# dfIris = pd.DataFrame(data = np.c_[iris['data'], iris['target']], columns =
iris['feature_names'] + ['target'])
dfIris = pd.DataFrame(data = np.c_[X, y], columns = iris['feature_names'] +
['target'])
dfIris
```

Out[5]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2.0
146	6.3	2.5	5.0	1.9	2.0
147	6.5	3.0	5.2	2.0	2.0
148	6.2	3.4	5.4	2.3	2.0
149	5.9	3.0	5.1	1.8	2.0

150 rows × 5 columns

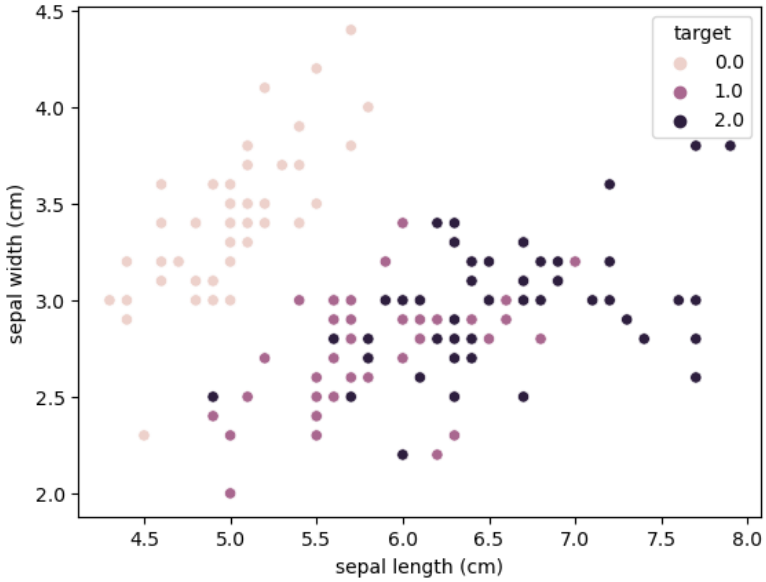
In [6]:
```
print(iris['target_names'])
['setosa' 'versicolor' 'virginica']
```

In [7]:
```
print(dfIris['target'].unique())
[0. 1. 2.]
```

- Покажемо на графіках пари відповідних параметрів і значення цільовох функції.

In [8]:
```
sns.scatterplot(x = dfIris['sepal length (cm)'], y = dfIris['sepal width (cm)'],
hue = dfIris['target'])
#plt.legend(bbox_to_anchor=(1,1), loc="upper left")
```

Out[8]:
```
<AxesSubplot:xlabel='sepal length (cm)', ylabel='sepal width (cm)'>
```

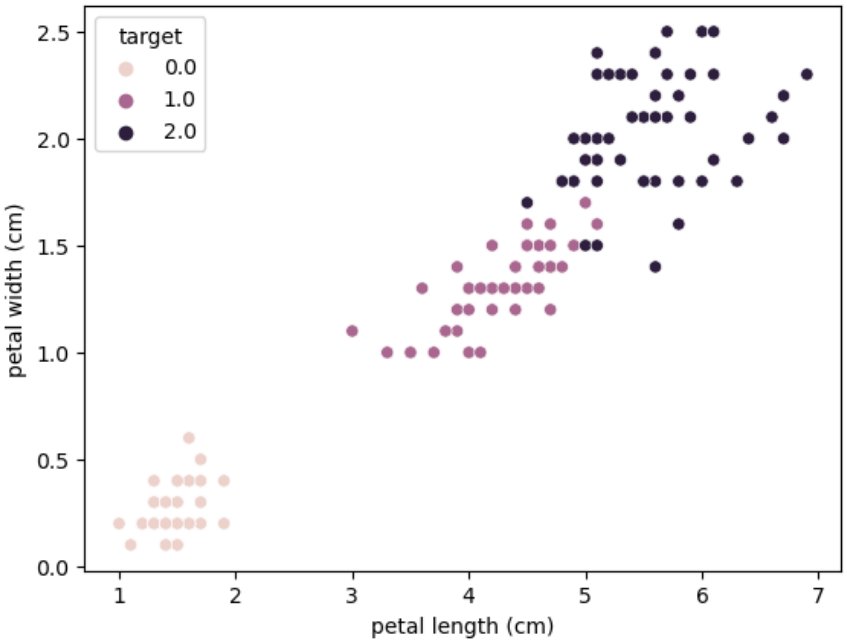


In [9]:

```
sns.scatterplot(x = dfIris['petal length (cm)'], y = dfIris['petal width (cm)'],
hue = dfIris['target'])
```

Out[9]:

```
<AxesSubplot:xlabel='petal length (cm)', ylabel='petal width (cm)'>
```



- Розділимо вибірку на навчальну і тестову.

In [10]:

```
X_train, X_test, y_train, y_test = \
                 train_test_split(X, y, test_size = 0.25, random_state = 1) #
75% training and 25% test
#print(X_train.shape, X_test.shape)
#print(y_train.shape, y_test.shape)
```

- Далі потрібно знайти відстані за відповідними формулами. Також створимо ще навчальний і тестовий датафрейм.

In [11]:

```
# знаходимо відстані між одним тестовим об'ктом (j) і всіма навчальними
(0,...,len(X_train))
def fun_distance(X_train, X_test, j):
    n_train = len(X_train)
    distances = [np.linalg.norm(X_train[i] - X_test[j]) for i in range(n_train)]
    return distances
```

In [12]:

```
def fun_df(X_train, X_test, y_train, y_test):
```

```
    print('df_test(наприклад, перший):')
    df_test = pd.DataFrame(data = np.c_[X_test, y_test], columns =
['x1','x2','x3','x4',] + ['y'])
    display(df_test.head(1))

    print('\ndf_train(перші 5 рядків):')
    df_train = pd.DataFrame(data = np.c_[X_train, y_train], columns =
['x1','x2','x3','x4',] + ['y'])
    display(df_train.head())

    return df_test, df_train

df_test, df_train = fun_df(X_train, X_test, y_train, y_test)
```

df_test(наприклад, перший):

	x1	x2	x3	x4	y
0	5.8	4.0	1.2	0.2	0.0

df_train(перші 5 рядків):

	x1	x2	x3	x4	y
0	6.5	2.8	4.6	1.5	1.0
1	6.7	2.5	5.8	1.8	2.0
2	6.8	3.0	5.5	2.1	2.0
3	5.1	3.5	1.4	0.3	0.0
4	6.0	2.2	5.0	1.5	2.0

- Для проведення зваженого голосування розрахуємо показник близькості **Q**.

In [13]:

```
def fun_Q(df_sort, clas):
    y = df_sort.query('`y` == @clas')

    if y.shape[0] == 0:
        Q = 0
    else:
        y['distances^2'] = (y['distances'])**2
        Q = 1 / y['distances^2']
        Q = Q.sum()

    #print('Q_{0:.0f} = {1}'.format(clas, Q))
    return clas, Q
```

- Тепер визначимо клас конкретного тестового об'єкта.

In [14]:

```
def fun_Class(distances, df_train, k):
    # df_test = pd.DataFrame(data = np.c_[X_test, y_test], columns =
['x1','x2','x3','x4',] + ['y'])

    # df_train = pd.DataFrame(data = np.c_[X_train, y_train], columns =
['x1','x2','x3','x4',] + ['y'])
    df_train['distances'] = distances
    display(df_train.head())

    # df_sort
    df_sort = df_train.sort_values(by=['distances'])
    display(df_sort.head())

    print('\nk сусідів =',k)
    df_sort = df_sort.head(k)
    display(df_sort.head())

    classes = df_train['y'].unique()
    classes.sort()
    #print('classes =', classes)
```

```
    Q, Class = [], []
    for i in classes:
        cl, q = fun_Q(df_sort, clas=i)
        Q.append(q)
        Class.append(cl)

    df_new = pd.DataFrame(data = np.c_[Class, Q], columns = ['class', 'Q'])
    display(df_new)

    answer = df_new[ df_new['Q'] == df_new['Q'].max() ]
    answer = answer['class'].values
    answer = int(answer[0])

    namees_class = iris['target_names']
    print('class prediction: ', answer, '-', namees_class[answer])

    return answer
```

- **Виконаємо всі дії за алгоритмом для кожного тестового об'єкта по черзі і продемонструємо їх.**

In [15]:

```
k_neighbours = 3
y_pred = []
for j_test in range(len(X_test)):
    print("----", j_test+1, " тестовий об`єкт ----")
    distances = fun_distance(X_train, X_test, j=j_test)
    y_pr = fun_Class(distances, df_train, k=k_neighbours)
    y_pred.append(y_pr)
    print("y_test =", y_test[j_test], '\n\n')
```

```
---- 1  тестовий об`єкт ----
```

	x1	x2	x3	x4	y	distances
0	6.5	2.8	4.6	1.5	1.0	3.896152
1	6.7	2.5	5.8	1.8	2.0	5.174940
2	6.8	3.0	5.5	2.1	2.0	4.909175
3	5.1	3.5	1.4	0.3	0.0	0.888819
4	6.0	2.2	5.0	1.5	2.0	4.405678

	x1	x2	x3	x4	y	distances
62	5.7	4.4	1.5	0.4	0.0	0.547723
8	5.7	3.8	1.7	0.3	0.0	0.556776
42	5.4	3.7	1.5	0.2	0.0	0.583095
13	5.5	3.5	1.3	0.2	0.0	0.591608
44	5.2	4.1	1.5	0.1	0.0	0.685565

```
k сусідів = 3
```

	x1	x2	x3	x4	y	distances
62	5.7	4.4	1.5	0.4	0.0	0.547723
8	5.7	3.8	1.7	0.3	0.0	0.556776
42	5.4	3.7	1.5	0.2	0.0	0.583095

	class	Q
0	0.0	9.500316
1	1.0	0.000000
2	2.0	0.000000

```
class prediction:  0 - setosa
```

y_test = 0

---- 2 тестовий об`єкт ----

	x1	x2	x3	x4	y	distances
0	6.5	2.8	4.6	1.5	1.0	2.184033
1	6.7	2.5	5.8	1.8	2.0	3.300000
2	6.8	3.0	5.5	2.1	2.0	3.223352
3	5.1	3.5	1.4	0.3	0.0	2.049390
4	6.0	2.2	5.0	1.5	2.0	2.249444

	x1	x2	x3	x4	y	distances
91	4.9	2.4	3.3	1.0	1.0	0.387298
39	5.0	2.3	3.3	1.0	1.0	0.387298
88	5.0	2.0	3.5	1.0	1.0	0.721110
106	5.7	2.6	3.5	1.0	1.0	0.793725
87	5.5	2.4	3.7	1.0	1.0	0.818535

k сусідів = 3

	x1	x2	x3	x4	y	distances
91	4.9	2.4	3.3	1.0	1.0	0.387298
39	5.0	2.3	3.3	1.0	1.0	0.387298
88	5.0	2.0	3.5	1.0	1.0	0.721110

	class	Q
0	0.0	0.00000
1	1.0	15.25641
2	2.0	0.00000

class prediction: 1 - versicolor
y_test = 1

[…]

---- 38 тестовий об`єкт ----

	x1	x2	x3	x4	y	distances
0	6.5	2.8	4.6	1.5	1.0	3.738984
1	6.7	2.5	5.8	1.8	2.0	4.998000
2	6.8	3.0	5.5	2.1	2.0	4.810405
3	5.1	3.5	1.4	0.3	0.0	0.173205
4	6.0	2.2	5.0	1.5	2.0	4.090232

	x1	x2	x3	x4	y	distances
50	5.2	3.5	1.5	0.2	0.0	0.141421
89	5.1	3.5	1.4	0.2	0.0	0.141421
12	5.1	3.4	1.5	0.2	0.0	0.141421
3	5.1	3.5	1.4	0.3	0.0	0.173205
95	5.0	3.4	1.5	0.2	0.0	0.223607

k сусідів = 3

	x1	x2	x3	x4	y	distances
50	5.2	3.5	1.5	0.2	0.0	0.141421
89	5.1	3.5	1.4	0.2	0.0	0.141421
12	5.1	3.4	1.5	0.2	0.0	0.141421

	class	Q
0	0.0	150.0
1	1.0	0.0
2	2.0	0.0

```
class prediction:  0 - setosa
y_test = 0
```

- ***Знайдемо точність - відповідність прогнозованих та реальних даних.***

In [16]:

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy (точність):", accuracy)
Accuracy (точність): 1.0
```

З використанням вбудованого алгоритму kNN з бібліотеки *sklearn* провести класифікацію об'єктів. Визначити кількість помилок.

In [17]:

```
# Масштабування функцій за допомогою StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [18]:

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```

Out[18]:

```
KNeighborsClassifier(n_neighbors=3)
```

In [19]:

```
y_pred = knn.predict(X_test)
y_pred
```

Out[19]:

```
array([0, 1, 1, 0, 2, 1, 2, 0, 0, 2, 1, 0, 2, 1, 1, 0, 1, 1, 0, 0, 1, 1,
       1, 0, 2, 1, 0, 0, 1, 2, 1, 2, 1, 2, 2, 0, 1, 0])
```

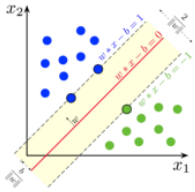
In [20]:

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy (точність):", accuracy)
Accuracy (точність): 1.0
```

Метод опорних векторів (SVM)

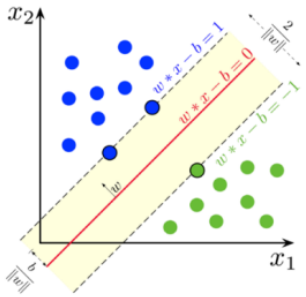
Лабораторна робота 3

Метод опорних векторів (SVM)
Постановка задачі



Розглянемо задачу бінарної класифікації, в якій об'єктам з $X = \mathbb{R}^2$ (об'єкти описуються **двома** числовими признаками): $\boldsymbol{X = \{x_1, x_2\}}$ відповідає один з двох класів $\boldsymbol{Y = \{-1, +1\}}$.
Нехай задана вибірка пар "об'єкт-відповідь": $(x_i, yi), i = 1 \dots N$.
Необхідно побудувати алгоритм класифікації $a(x): X \to Y$.

Розділяюча полоса

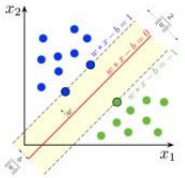


Етапи розв'язання

1. Провести первинну обробку даних вибірки:
   - Визначити дві ознаки $X = \{x_1, x_2\}$
   - Визначити два класи, з умовою $Y = \{-1, +1\}$
2. Розділити всю вибірку на навчальну та тестову
3. Побудувати алгоритм навчання на навчальній вибірці
4. Перевірити точність роботи алгоритму на тестовій вибірці
5. Порівняти результати з SVM з sklearn
6. Оформити результати у вигляді звіту.

2

Метод опорних векторів (SVM)
в задачах класифікації



1. Первинна обробка даних

$x = (x_1, x_2, \dots, x_n)$

$x_{n+1} = 1; w_0 = -b$

$[\langle x_i, w\rangle - b]_{i=1\dots n} \to [\langle x_i, w\rangle]_{i=1\dots 3}$

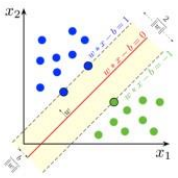
Radius_mean	Texture_mean	Perimeter_mean	Area_mean	Diagnosis
17.99	10.38	122.8	100	1
20.57	17.77	132.9	132	1
19.69	21.25	130	120	1
11.42	20.38	77.58	386.	1
20.29	14.34	135.1	129	1
12.45	15.7	82.57	477.	1
18.25	19.98	119.6	104	1
13.71	20.83	90.2	577.	1
13	21.82	87.5	519.	1
12.46	24.04	83.97	475.	1
16.02	23.24	102.7	797.	1
15.78	17.89	103.6	78	1
14.61	15.69	92.68	664.	0
12.76	13.37	82.29	504.	0
11.54	10.72	73.73	409.	0
8.597	18.6	54.09	221.	0
12.49	16.85	79.19	481.	0
12.18	14.08	77.25	461.	0
18.22	18.87	118.7	103	1
9.042	18.9	60.07	244.	0

$y = \{-1, 1\}$

1.1. Визначити 2 головних признаки за якими буде проводитись класифікація
1.2. Додати ще один стовбець до матриці ознак зі значеннями 1
1.3. Встановити значення цільового вектору +1 та -1

3

Метод опорних векторів (SVM) в задачах класифікації



Розділення вибірки на навчальну та тестову

$x_{train} = [0 \dots \ell]; \qquad x_{test} = [\ell \dots N]$

Побудова алгоритму навчання

Відступ $M_i(w) = yi\langle x_i, w\rangle$

Задача оптимізації $f(w) = \frac{1}{\ell}\sum_{i=1}^{\ell}\left[\frac{1}{2}\|w\| + C(1 - y_i\langle w, x_i\rangle)_+\right]$

Розрахунок наступного значення ваг w за методом градієнтного спуску:

$$w_i = w_i - \eta\frac{1}{\ell}\nabla f(w)$$

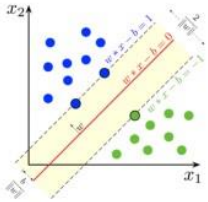
Похідна від функціоналу якості

$$\nabla f(w) = \frac{1}{\ell}\sum_{i=1}^{\ell}\begin{cases} w, & if\ \max(1 - y_i\langle w, x_i\rangle)_+ = 0 \\ w - Cy_ix_i, & otherwise \end{cases}$$



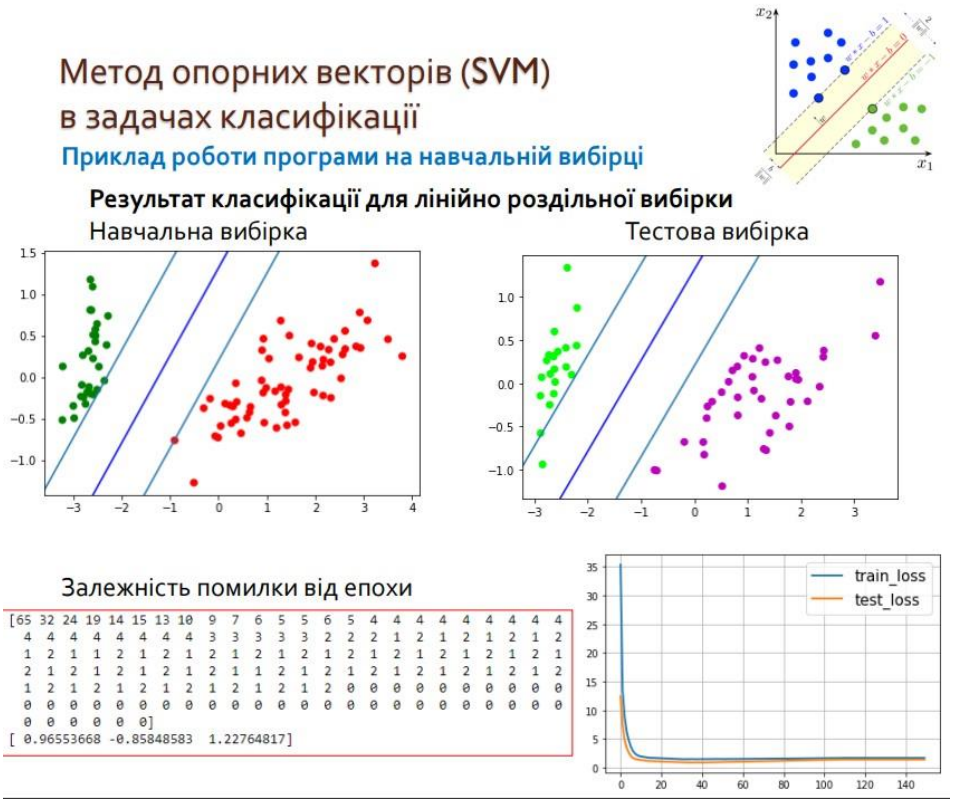
Визначити оптимальні значення параметра регуляризації C та швидкості навчання η для навчання на навчальній вибірці

Метод опорних векторів (SVM)



Етапи розв'язання

1. Імпортувати вибірку для проведення навчання
2. Розділити всю вибірку на навчальну та тестову
3. Провести підготовку даних до класифікації
   - визначити дві головних ознаки за якими буде проводитись класифікація додати ще один стовбчик до матриці ознак та встановити всі значення рівним 1
   - встановити значення цільового вектора +1 та -1
4. Побудувати алгоритм навчання на навчальній вибірці
   - встановити значення кроку навчання, коефіцієнта змінення ваг
   - визначити умову завершення навчання (встановити кількість епох навчання або мінімально допустиму точність алгоритму)
   - в залежності від величини зазору змінювати ваги за допомогою градієнта функції втрат Q
   - рахувати кількість помилок (неправильно класифікованих об'єктів) у процесі навчання
5. Подати графічно результат класифікації для навчальної вибірки та залежність помилок від номеру епохи навчання
6. Перевірити точність роботи алгоритму на тестовій вибірці
7. Подати графічно результат класифікації для тестової вибірки
8. Порівняти результати з SVM з sklearn
9. Оформити результати у вигляді звіту.



- **Імпортуємо потрібні бібліотеки**:

In [1]:

```
import numpy as np
import pandas as pd

import warnings
warnings.filterwarnings('ignore')

import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.lines as mlines
plt.rcParams['figure.figsize'] = (8,6)
%matplotlib inline

from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn import metrics
```

- **Визначаємо функції, що реалізовують Метод опорних векторів (svm):**

In [2]:

```
 # функція для зображення лінії
def newline(p1, p2, color=None):
    ax = plt.gca()
    xmin, xmax = ax.get_xbound()

    if(p2[0] == p1[0]):
        xmin = xmax = p1[0]
        ymin, ymax = ax.get_ybound()
    else:
        ymax = p1[1]+(p2[1]-p1[1])/(p2[0]-p1[0])*(xmax-p1[0])
        ymin = p1[1]+(p2[1]-p1[1])/(p2[0]-p1[0])*(xmin-p1[0])

    l = mlines.Line2D([xmin,xmax], [ymin,ymax], color=color)
    ax.add_line(l)
    return l
```

In [3]:

```
def add_bias_feature(a):
```

```
    a_extended = np.zeros((a.shape[0],a.shape[1]+1))
    a_extended[:,:-1] = a
    a_extended[:,-1] = int(1)
    return a_extended
```

In [4]:

```
class CustomSVM(object):
    __class__ = "CustomSVM"
    __doc__ = """
    Це реалізація алгоритму класифікації SVM (тільки для бінарної класифікації)

    # Параметри
    etha: float(default - 0.01)
        Швидкість навчання, крок градієнта

    alpha: float, (default - 0.1)
        Параметр регуляризації в 0.5*alpha*||w||^2

    epochs: int, (default - 200)
        Кількість епох навчання
    """

    def __init__(self, etha=0.01, alpha=0.1, epochs=200):
        self._epochs = epochs
        self._etha = etha
        self._alpha = alpha
        self._w = None
        self.history_w = []
        self.train_errors = None
        self.val_errors = None
        self.train_loss = None
        self.val_loss = None

    def fit(self, X_train, Y_train, X_val, Y_val, verbose=False): #arrays: X; Y
=-1,1

        if len(set(Y_train)) != 2 or len(set(Y_val)) != 2:
            raise ValueError("Кількість класів в Y не дорівнює 2!")

        X_train = add_bias_feature(X_train)
        X_val = add_bias_feature(X_val)
        self._w = np.random.normal(loc=0, scale=0.05, size=X_train.shape[1])
        self.history_w.append(self._w)
        train_errors = []
        val_errors = []
        train_loss_epoch = []
        val_loss_epoch = []

        for epoch in range(self._epochs):
            tr_err = 0
            val_err = 0
            tr_loss = 0
            val_loss = 0
            for i,x in enumerate(X_train):
                margin = Y_train[i]*np.dot(self._w,X_train[i])
                if margin >= 1: # классифицируем верно
                    self._w = self._w -
self._etha*self._alpha*self._w/self._epochs
                    tr_loss += self.soft_margin_loss(X_train[i],Y_train[i])
                else: # классифицируем неверно или попадаем на полосу разделения
при 0<m<1
                    self._w = self._w +\
                    self._etha*(Y_train[i]*X_train[i] -
self._alpha*self._w/self._epochs)
                    tr_err += 1
                    tr_loss += self.soft_margin_loss(X_train[i],Y_train[i])
```

```
                self.history_w.append(self._w)
            for i,x in enumerate(X_val):
                val_loss += self.soft_margin_loss(X_val[i], Y_val[i])
                val_err += (Y_val[i]*np.dot(self._w,X_val[i])<1).astype(int)
            if verbose:
                print('epoch {}. Errors={}. Mean Hinge_loss={}'\
                      .format(epoch,err,loss))
            train_errors.append(tr_err)
            val_errors.append(val_err)
            train_loss_epoch.append(tr_loss)
            val_loss_epoch.append(val_loss)
        self.history_w = np.array(self.history_w)
        self.train_errors = np.array(train_errors)
        self.val_errors = np.array(val_errors)
        self.train_loss = np.array(train_loss_epoch)
        self.val_loss = np.array(val_loss_epoch)

    def predict(self, X:np.array) -> np.array:
        y_pred = []
        X_extended = add_bias_feature(X)
        for i in range(len(X_extended)):
            y_pred.append(np.sign(np.dot(self._w,X_extended[i])))
        return np.array(y_pred)

    def hinge_loss(self, x, y):
        return max(0,1 - y*np.dot(x, self._w))

    def soft_margin_loss(self, x, y):
        return self.hinge_loss(x,y)+self._alpha*np.dot(self._w, self._w)
```

- **Імпортуємо вибірку для проведення навчання**

In [5]:

```
iris = load_iris()
X = iris.data
y = iris.target
```

In [6]:

```
print(np.shape(X))
print(np.shape(y))
(150, 4)
(150,)
```

In [7]:

```
dfIris = pd.DataFrame(data = np.c_[X, y], columns = iris['feature_names'] +
['target'])
dfIris
```

Out[7]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2.0
146	6.3	2.5	5.0	1.9	2.0
147	6.5	3.0	5.2	2.0	2.0
148	6.2	3.4	5.4	2.3	2.0
149	5.9	3.0	5.1	1.8	2.0

150 rows × 5 columns

In [8]:

```
print(dfIris['target'].unique())
```

```
[0. 1. 2.]
```

- **Проводимо підготовку даних до класифікації:**

In [9]:

```
pca = PCA(n_components=2)
X = pca.fit_transform(X)

y = (y > 0).astype(int)*2-1 # [0,1,2] --> [False,True,True] --> [0,1,1] -->
[0,2,2] --> [-1,1,1]
```

In [10]:

```
dfIris2 = pd.DataFrame(data = np.c_[X, y], columns = ['x1', 'x2'] + ['y'])
dfIris2
```

Out[10]:

	x1	x2	y
0	-2.684126	0.319397	-1.0
1	-2.714142	-0.177001	-1.0
2	-2.888991	-0.144949	-1.0
3	-2.745343	-0.318299	-1.0
4	-2.728717	0.326755	-1.0
...	...	...	...
145	1.944110	0.187532	1.0
146	1.527167	-0.375317	1.0
147	1.764346	0.078859	1.0
148	1.900942	0.116628	1.0
149	1.390189	-0.282661	1.0

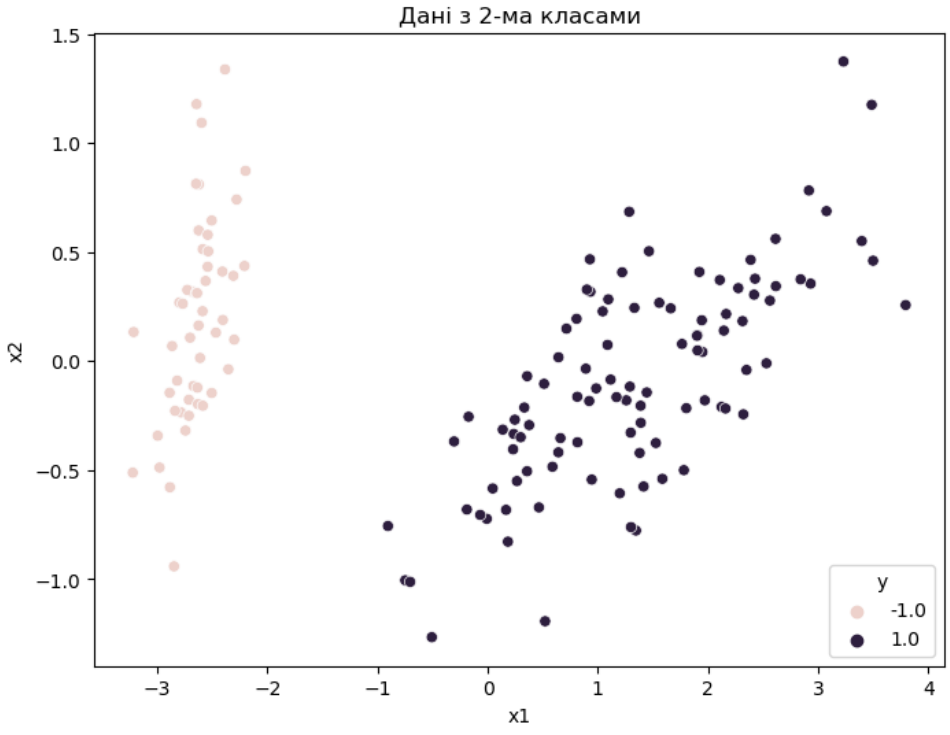
150 rows × 3 columns

In [11]:

```
sns.scatterplot(x = dfIris2['x1'], y = dfIris2['x2'], hue = dfIris2['y'])
plt.title('Дані з 2-ма класами')
#plt.legend(bbox_to_anchor=(1,1), loc="upper left")
```

Out[11]:

```
Text(0.5, 1.0, 'Дані з 2-ма класами')
```



- **Розділяємо всю вибірку на навчальну та тестову. Навчаємо алгоритм на навчальній вибірці.**

In [12]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=2023)
```
In [13]:
```
# блок ініціалізації та навчання
svm = CustomSVM(etha=0.005, alpha=0.006, epochs=150)
svm.fit(X_train, y_train, X_test, y_test)
```

- **Подаємо графічно результат класифікації для навчальної вибірки та залежність помилок від номеру епохи навчання.**

In [14]:
```
df_train = pd.DataFrame(data = np.c_[X_train, y_train], columns = ['x1', 'x2'] +
['y'])
display(df_train.head(5))
```

	x1	x2	y
0	1.764346	0.078859	1.0
1	-2.310256	0.391346	-1.0
2	-0.173925	-0.254854	1.0
3	1.971531	-0.179728	1.0
4	0.136429	-0.314032	1.0

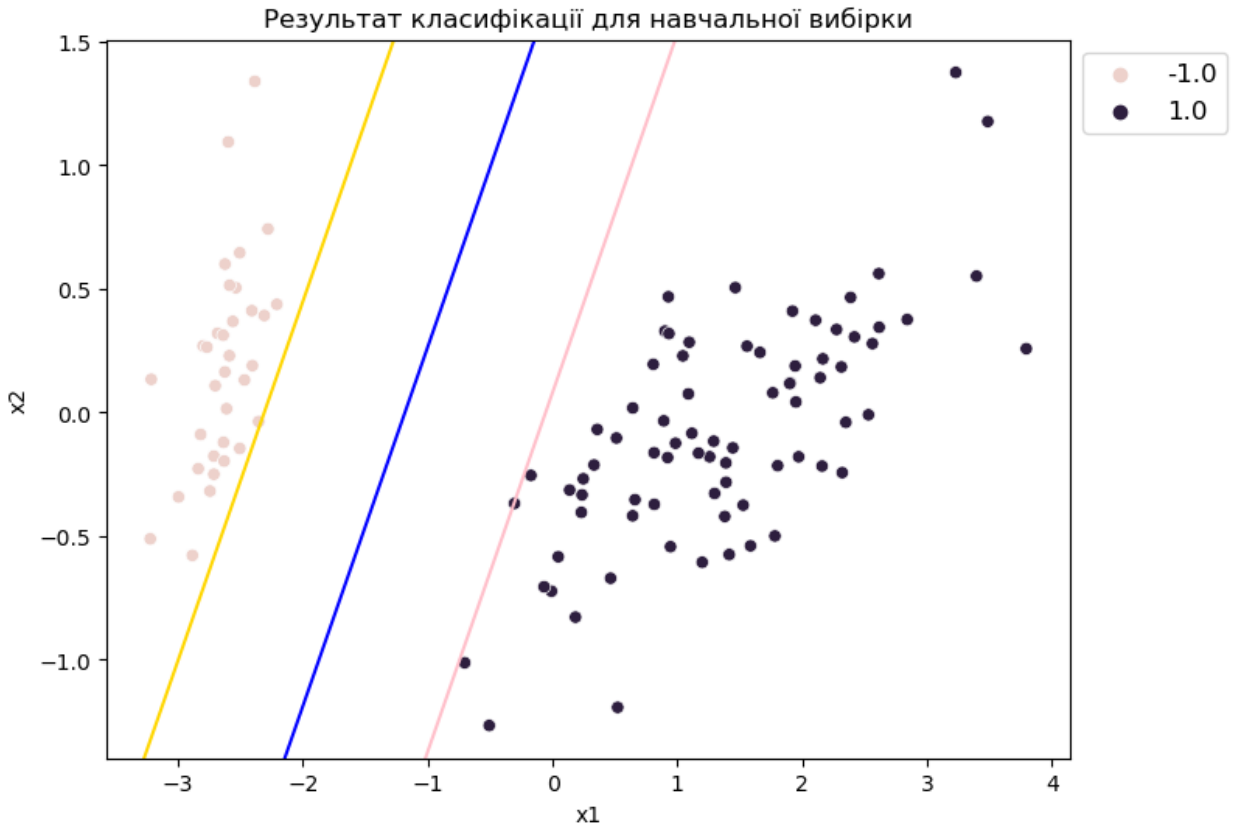
In [15]:
```
sns.scatterplot(x = df_train['x1'], y = df_train['x2'], hue = df_train['y'])

# в w0*x_i[0]+w1*x_i[1]+w2*1=0 по черзі
# підставляємо x_i[0]=0, x_i[1]=0
newline([0,-svm._w[2]/svm._w[1]],[-svm._w[2]/svm._w[0],0], 'blue')

# w0*x_i[0] + w1*x_i[1] + w2*1 = 1
newline([0,  1 / svm._w[1]-svm._w[2]/svm._w[1]], [ 1 / svm._w[0]-
svm._w[2]/svm._w[0], 0], 'pink')

# w0*x_i[0] + w1*x_i[1] + w2*1 = -1
newline([0, -1 / svm._w[1]-svm._w[2]/svm._w[1]], [-1 / svm._w[0]-
svm._w[2]/svm._w[0], 0], 'gold')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Результат класифікації для навчальної вибірки" )
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
plt.show()
```

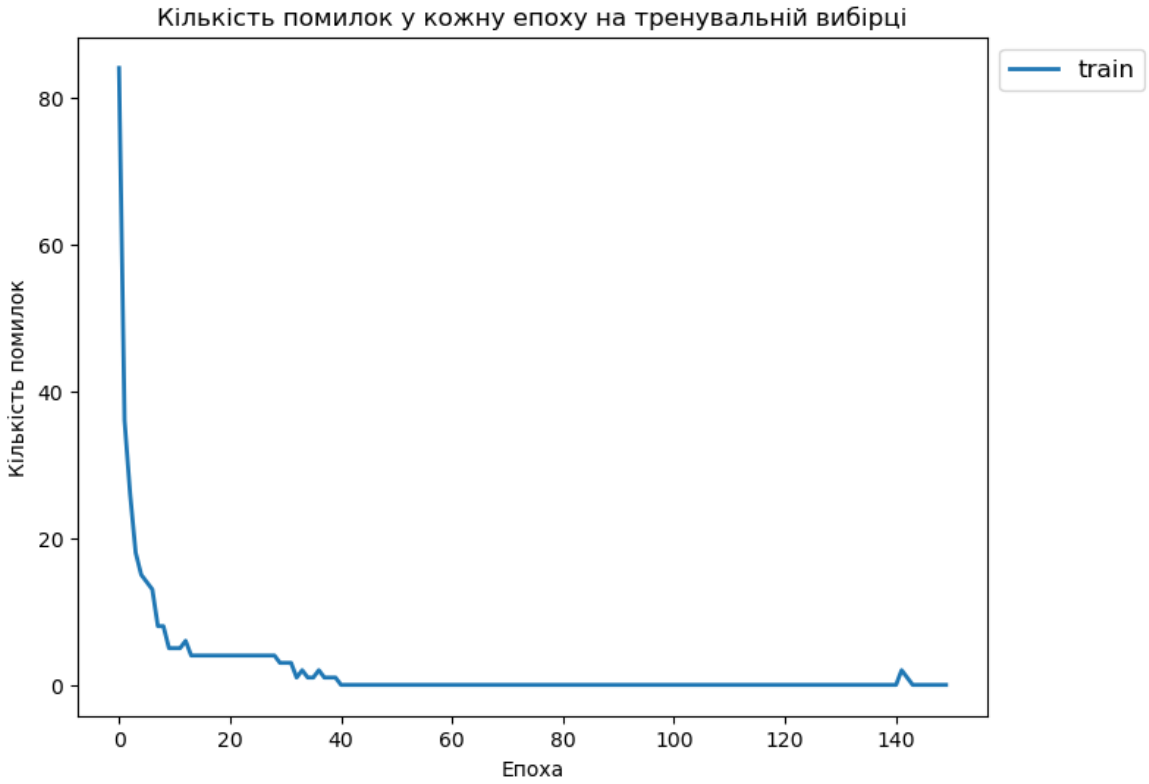


In [16]:

```
print('Кількість помилок у кожну епоху:\n', svm.train_errors, '\n')
print('Значення ваг або коефіцієнтів для прямої w0*x_i[0]+w1*x_i[1]+w2=0 :\n',
svm._w)

plt.plot(svm.train_errors, linewidth=2, label='train')
plt.xlabel("Епоха")
plt.ylabel("Кількість помилок")
plt.title("Кількість помилок у кожну епоху на тренувальній вибірці" )
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
plt.show()
Кількість помилок у кожну епоху:
 [84 36 26 18 15 14 13  8  8  5  5  5  6  4  4  4  4  4  4  4  4  4  4  4
  4  4  4  4  4  3  3  3  1  2  1  1  2  1  1  1  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  2  1  0
  0  0  0  0  0  0]

Значення ваг або коефіцієнтів для прямої w0*x_i[0]+w1*x_i[1]+w2=0 :
 [ 0.88713539 -0.61137468  1.0493689 ]
```



- **Перевіряємо точність роботи алгоритму на тестовій вибірці:**

In [17]:

```
# Прогнозуємо значення на тестовій вибірці
y_predict = svm.predict(X_test)
```

In [18]:

```
# Вимірюємо точність прогнозів
print("Accuracy:", metrics.accuracy_score(y_test, y_predict))
#print("Accuracy score: %.3f" %metrics.accuracy_score(y_test, y_predict))
Accuracy: 1.0
```

- **Подаємо графічно результат класифікації для тестової вибірки**

In [19]:

```
df_test = pd.DataFrame(data = np.c_[X_test, y_test], columns = ['x1', 'x2'] +
['y'])
display(df_test.head(5))
```

	x1	x2	y
0	2.123609	-0.209729	1.0
1	0.714853	0.149056	1.0
2	0.299001	-0.348898	1.0
3	2.932587	0.355500	1.0
4	1.332024	0.244441	1.0

In [20]:

```
sns.scatterplot(x = df_test['x1'], y = df_test['x2'], hue = df_test['y'])
#d = {-1:'yellow', 1:'blue'}
#plt.scatter(X_test[:,0], X_test[:,1], c=[d[y] for y in y_test], label = d)

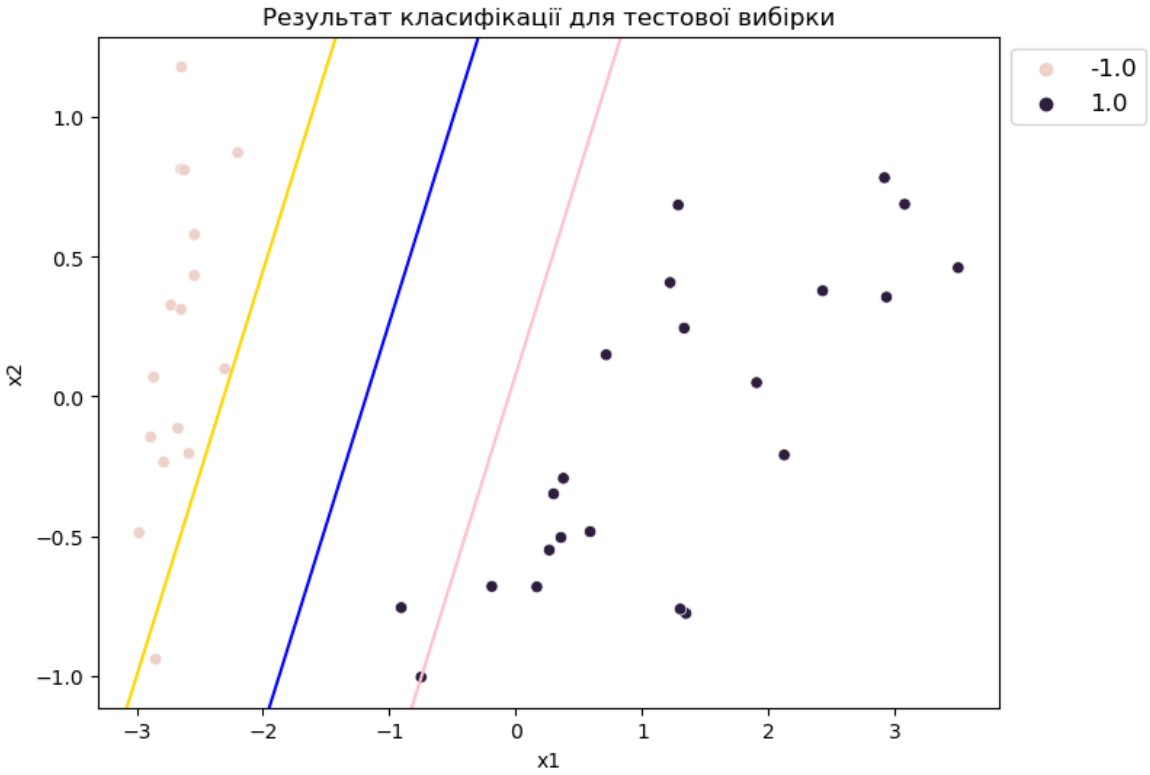
# в w0*x_i[0] + w1*x_i[1] + w2*1 = 0 по черзі
# підставляємо x_i[0] = 0, x_i[1] = 0
newline([0, -svm._w[2] / svm._w[1]], [-svm._w[2]/svm._w[0], 0], 'blue')

# w0*x_i[0] + w1*x_i[1] + w2*1 = 1
newline([0,  1 / svm._w[1]-svm._w[2]/svm._w[1]], [ 1 / svm._w[0]-
svm._w[2]/svm._w[0], 0], 'pink')

# w0*x_i[0] + w1*x_i[1] + w2*1 = -1
```

```
newline([0, -1 / svm._w[1]-svm._w[2]/svm._w[1]], [-1 / svm._w[0]-
svm._w[2]/svm._w[0], 0], 'gold')

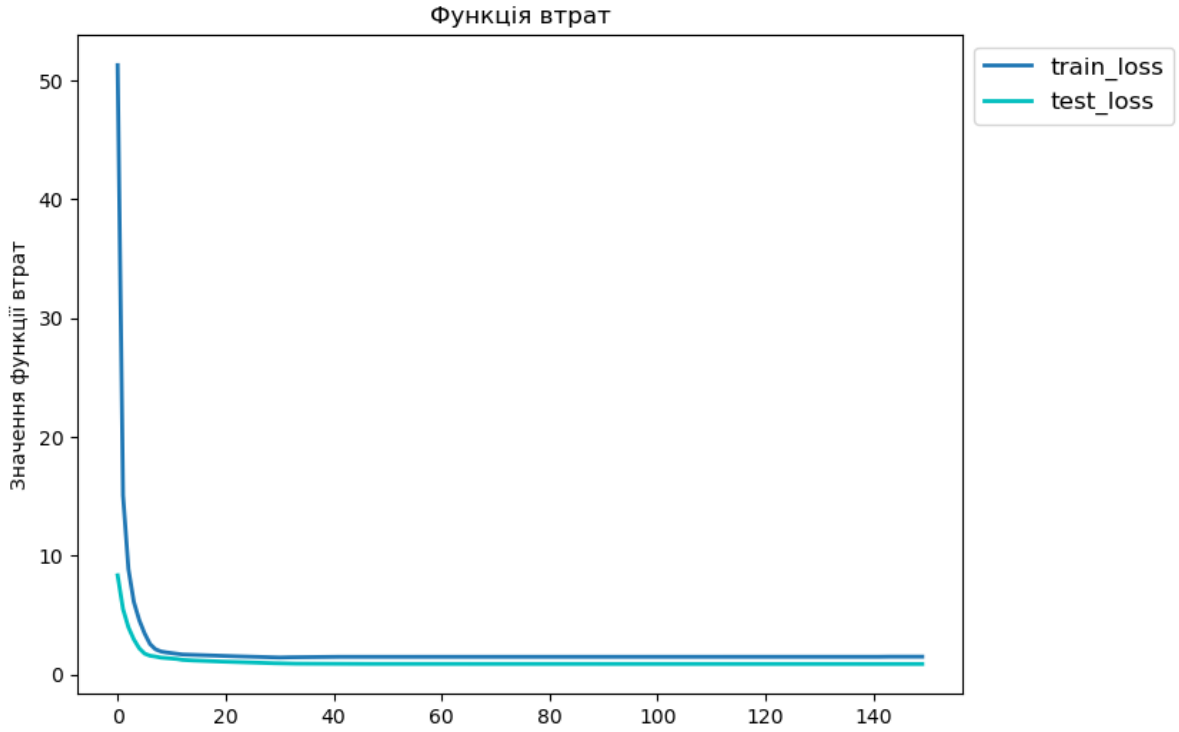
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Результат класифікації для тестової вибірки" )
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
plt.show()
```



- Побудуємо функції втрат навчальної та тестовоої вибірки.

In [21]:

```
plt.plot(svm.train_loss, linewidth=2, label='train_loss')
plt.plot(svm.val_loss, linewidth=2, label='test_loss', c = 'c')
plt.ylabel("Значення функції втрат")
plt.title("Функція втрат" )
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
plt.show()
```



- **Порівняємо результати з SVM з sklearn.**

In [22]:

```
#Import svm model
from sklearn import svm

# Створюємо svm Класифікатор
clf = svm.SVC(kernel='linear') # Linear Kernel

# Навчаємо модель за допомогою навчальних наборів
clf.fit(X_train, y_train)

# Прогнозуємо значення цільової змінної для тестового набору даних
y_pred = clf.predict(X_test)
```

In [23]:

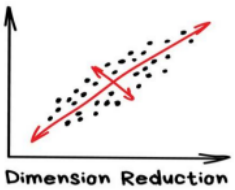
```
# Вимірюємо точність прогнозів
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
#print("Accuracy score: %.3f" %metrics.accuracy_score(y_test, y_predict))
Accuracy: 1.0
```

Отже, результати співпадають.

Метод головних компонент (PCA)

Лабораторна робота 4

Метод головних компонент (PCA)
Постановка задачі



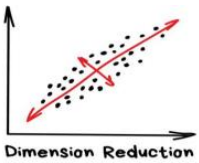
Провести реалізацію алгоритму PCA для зменшення розмірності вибірки до двох.

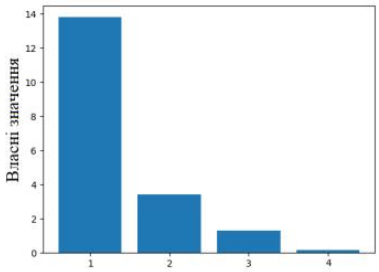
Етапи розв'язання

1. Імпортувати дані з кількістю ознак більше трьох.
2. Провести центрування та за необхідності нормалізацію даних
3. Побудувати матрицю коваріації.
4. Розрахувати власні вектори та власні значення матриці
5. Відсортувати власні вектори в порядку спадання власних значень Візуалізувати власні значення
6. Спроектувати дані на два перші власні вектори
7. Візуалізувати отриману дво-вимірну вибірку
8. Порахувати відсоток втраченої інформації
9. Розрахувати принципові компоненти за допомогою вбудованої функції PCA з кількістю компонентів 2 з бібліотеки sklearn.
10. Порівняти результати роботи власного алгоритму з результатами функції PCA з бібліотеки sklearn
11. Візуалізувати різницю в отриманих вибірках
12. Оформити результати у вигляді звіту.

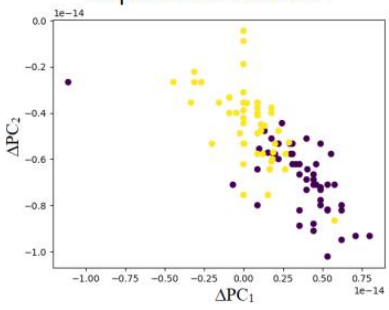
2

Метод головних компонент (PCA)
Приклад подання результатів

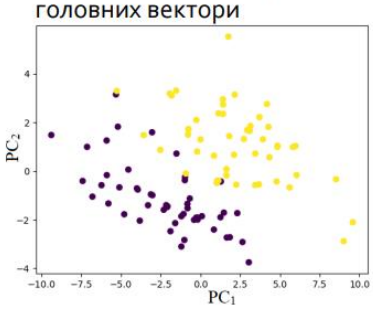




Порівняння з sclearn



Проекція даних на два головних вектори



Відсоток втраченої інформації

$$\Delta I = 1 - \frac{\lambda_1 + \lambda_2}{\sum_{i=1}^{n} \lambda_i} \times 100\% \simeq 7.9\%$$

- **Імпортуємо потрібні бібліотеки:**

In [1]:

```
#!pip install seaborn
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

In [2]:

```
import warnings
warnings.filterwarnings('ignore')
```

In [3]:

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
#from sklearn import metrics
```

1. Імпортувати дані з кількістю ознак більше трьох.

In [4]:

```
iris = load_iris()
features = iris.data
target = iris.target
```

In [5]:

```
print(np.shape(features))
print(np.shape(target))
(150, 4)
(150,)
```

In [6]:

```
dfIris = pd.DataFrame(data = np.c_[features, target], columns =
iris['feature_names'] + ['target'])
dfIris
```

Out[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2.0
146	6.3	2.5	5.0	1.9	2.0
147	6.5	3.0	5.2	2.0	2.0
148	6.2	3.4	5.4	2.3	2.0
149	5.9	3.0	5.1	1.8	2.0

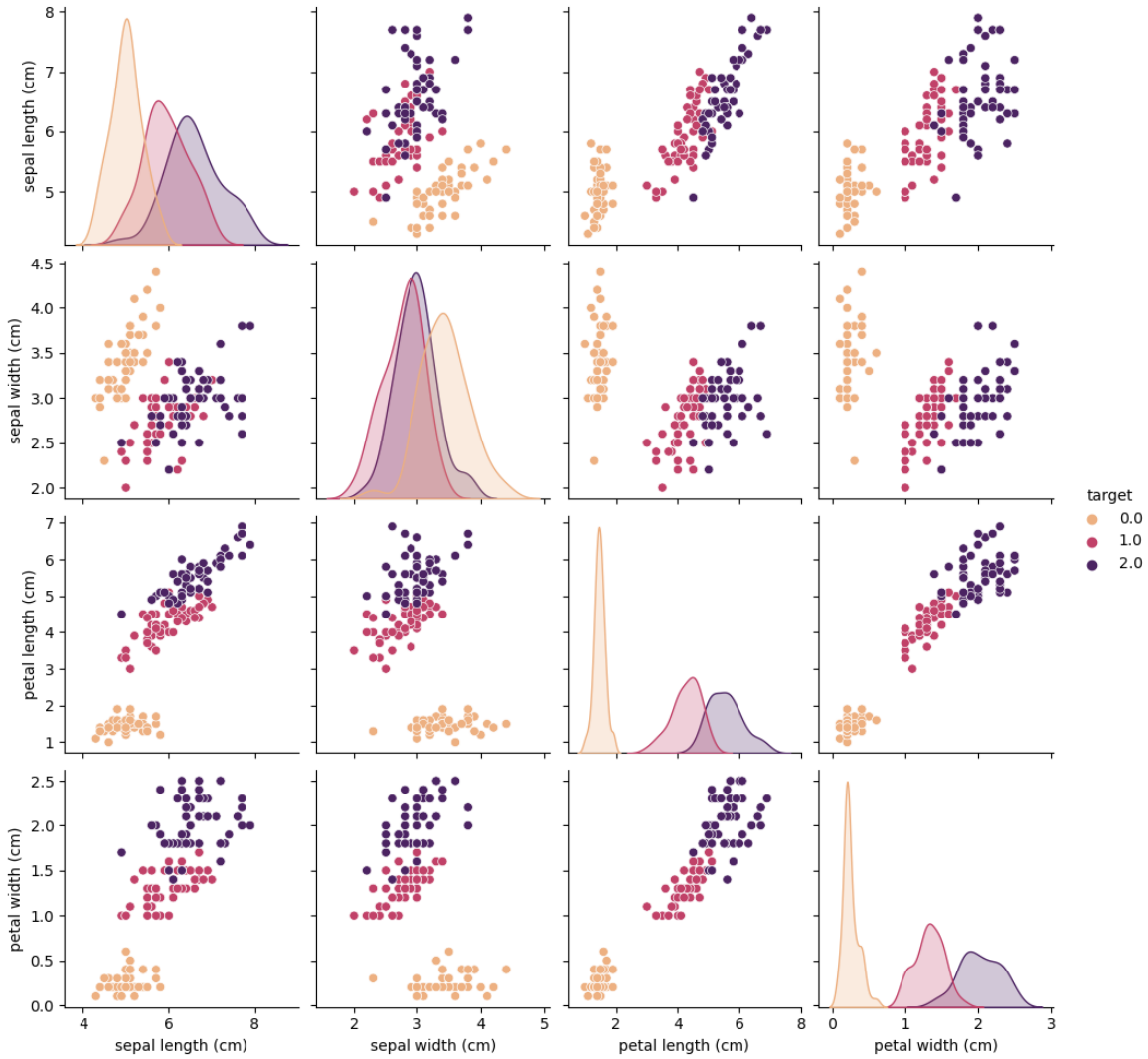
150 rows × 5 columns

In [7]:

```
print(dfIris['target'].unique())
[0. 1. 2.]
```

In [8]:

```
sns.pairplot(dfIris, hue ='target', palette = "flare")
plt.show()
```



Оскільки перший клас добре виділяється від інших двох залишимо у вибірці лише дані для другого та третього класів. Отримуємо вибірку X = [ℓ×n], де ℓ – кількість об'єктів, n – кількість ознак (для даного прикладу n = 4).

In [9]:

```
dfIris2 = dfIris.query("`target` != 0")
dfIris2
```

Out[9]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
50	7.0	3.2	4.7	1.4	1.0
51	6.4	3.2	4.5	1.5	1.0
52	6.9	3.1	4.9	1.5	1.0
53	5.5	2.3	4.0	1.3	1.0
54	6.5	2.8	4.6	1.5	1.0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2.0
146	6.3	2.5	5.0	1.9	2.0
147	6.5	3.0	5.2	2.0	2.0
148	6.2	3.4	5.4	2.3	2.0
149	5.9	3.0	5.1	1.8	2.0

100 rows × 5 columns

In [10]:

```
dfIris2['target'].unique()
```

Out[10]:

```
array([1., 2.])
```

In [11]:

```
X = np.array((dfIris2.drop(columns='target')).values)
X[:5]
```

Out[11]:

```
array([[7. , 3.2, 4.7, 1.4],
       [6.4, 3.2, 4.5, 1.5],
       [6.9, 3.1, 4.9, 1.5],
       [5.5, 2.3, 4. , 1.3],
       [6.5, 2.8, 4.6, 1.5]])
```

In [12]:

```
#y = np.array((dfIris2.drop(columns=['sepal length (cm)','sepal width
(cm)','petal length (cm)','petal width (cm)'])).values)
#y
```

2. Провести центрування та за необхідності нормалізацію даних.

In [13]:

```
from sklearn.preprocessing import StandardScaler
# масштабування даних
x_scaled = StandardScaler().fit_transform(X)
```

In [14]:

```
X[0]
```

Out[14]:

```
array([7. , 3.2, 4.7, 1.4])
```

In [15]:

```
x_scaled[0]
```

Out[15]:

```
array([ 1.11900931,  0.99068792, -0.25077906, -0.65303909])
```

In [16]:

```
x_matrix = np.array([x_scaled[:, 0], x_scaled[:, 1], x_scaled[:, 2], x_scaled[:,
3]])
```

3. Побудувати матрицю коваріації.

In [17]:

```
covmat = np.cov(x_matrix)
covmat
```

Out[17]:

```
array([[1.01010101, 0.55944934, 0.83684713, 0.59970648],
       [0.55944934, 1.01010101, 0.52505282, 0.57192176],
       [0.83684713, 0.52505282, 1.01010101, 0.83166424],
       [0.59970648, 0.57192176, 0.83166424, 1.01010101]])
```

In [18]:

```
np.shape(covmat)
```

Out[18]:

```
(4, 4)
```

4. Розрахувати власні вектори та власні значення матриці.

In [19]:

```
w, v = np.linalg.eig(covmat)
print('Власні значення (вектор):\n', w)
Власні значення (вектор):
 [2.98778909 0.08206412 0.40983199 0.56071884]
```

In [20]:

```
print('Власні вектори(масив):\n', v)
Власні вектори(масив):
 [[-0.5071303  -0.4623842  -0.69307315  0.22061124]
 [-0.43474966  0.13624797 -0.05558682 -0.88844771]
 [-0.54369524  0.74828559  0.01943532  0.37958719]
 [-0.5081408  -0.45574777  0.71845806  0.13380926]]
```

Відповідність власних значень та векторів:

- w[0] відповідає 0-му стовпчику v (v[:,0]),
- w[1] відповідає 1-му стовпчику v (v[:,1]),
- w[i] відповідає і-му стовпчику v (v[:,i]).

In [21]:

```
w[0]
```

Out[21]:

```
2.9877890906474667
```

In [22]:

```
v[:,0]
```

Out[22]:

```
array([-0.5071303 , -0.43474966, -0.54369524, -0.5081408 ])
```

Перевірка $ Au = w_1 u:$

In [23]:

```
u = v[:, 1]
print('u ', u)

w1 = w[1]
print('w1 ', w1, '\n')

print('Au \t', np.dot(covmat, u))
print('w1 u \t', w1*u)
```

```
u  [-0.4623842   0.13624797  0.74828559 -0.45574777]
w1  0.08206411695786031

Au       [-0.03794515  0.01118107  0.0614074  -0.03740054]
w1 u     [-0.03794515  0.01118107  0.0614074  -0.03740054]
```

5. Відсортувати власні вектори в порядку спадання власних значень. Візуалізувати власні значення.

In [24]:

```
data = [('1', w[0]),
        ('2', w[1]),
        ('3', w[2]),
        ('4', w[3])]
df = pd.DataFrame(data, columns =['№ ознаки', 'Власне значення'])
df
```

Out[24]:

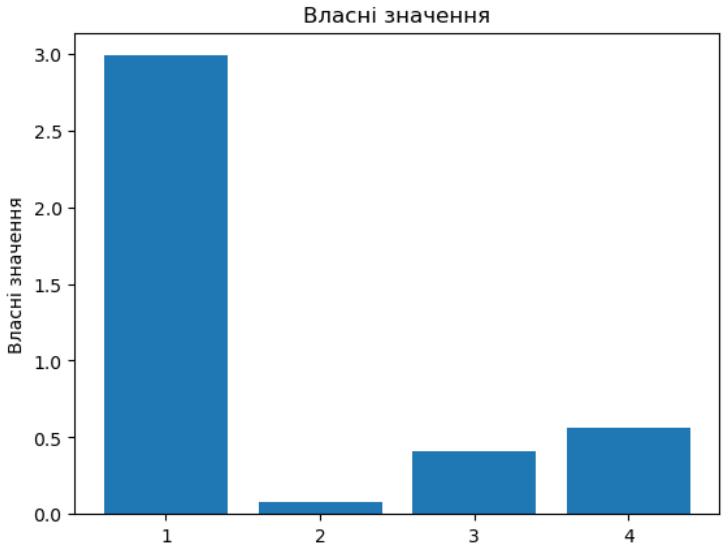
	№ ознаки	Власне значення
0	1	2.987789
1	2	0.082064
2	3	0.409832
3	4	0.560719

In [25]:

```
plt.bar(df['№ ознаки'], df['Власне значення'])
plt.ylabel('Власні значення')
plt.title('Власні значення')
```

Out[25]:

```
Text(0.5, 1.0, 'Власні значення')
```



In [26]:

```
df_sort = df.sort_values(by=['Власне значення'], ascending=False)
```

```
display(df_sort)
plt.bar(df_sort['№ ознаки'], df_sort['Власне значення'])
plt.ylabel('Власні значення')
plt.title('Власні значення (відсортовані за спаданням)')
```

	№ ознаки	Власне значення
0	1	2.987789
3	4	0.560719
2	3	0.409832
1	2	0.082064

Out[26]:

```
Text(0.5, 1.0, 'Власні значення (відсортовані за спаданням)')
```



6. Спроектувати дані на два перші власні вектори. Встановимо кількість головних компонент $nPC = 2$. Проектуємо дані на перші два власні вектори матриці коваріації, що характеризуються найбільшими власними значеннями.

Множимо вихідну матрицюданих розміру $[\ell \times n]$ на транспоновану матрицю двох власних векторів розміру $[n \times nPC]$ для отримання нової матриці даних розміром $[\ell \times nPC]$. Візуалізуємо отримані дані.

In [27]:

```
np.shape(x_scaled) # [ℓ × n]
```

Out[27]:

```
(100, 4)
```

In [28]:

```
index1, index2 = (df_sort['№ ознаки'].head(2)).values
index1, index2 = int(index1), int(index2)
print(index1, index2)
```

```
1 4
```

In [29]:

```
eigenvectors = np.array([v[index1-1], v[index2-1]])
vT = eigenvectors.transpose()
print(np.shape(vT)) # [n × nPC]
print(vT)
```

```
(4, 2)
[[-0.5071303  -0.5081408 ]
 [-0.4623842  -0.45574777]
 [-0.69307315  0.71845806]
 [ 0.22061124  0.13380926]]
```

In [30]:

```
x_new = np.dot(x_scaled, vT)
np.shape(x_new) # [ℓ × nPC]
```

Out[30]:

```
(100, 2)
```

7. Візуалізувати отриману дво-вимірну вибірку.

In [31]:

```
lab = dfIris2['target'].unique()
plt.plot(x_new, 'o', markersize = 5,
         #markerfacecolor='white',
         #markeredgecolor='gray',
         markeredgewidth = 2, label=lab)

plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('Проекція даних на два головних вектори')
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[31]:

```
<matplotlib.legend.Legend at 0x220a26b86a0>
```



8. Порахувати відсоток втраченої інформації, використовуючи власні значення матриці коваріацій при зменшенні розмірності з чотирьох ознак до двох:

In [32]:

```
df_sort
```

Out[32]:

	№ ознаки	Власне значення
0	1	2.987789
3	4	0.560719
2	3	0.409832
1	2	0.082064

In [33]:

```
l1 = df_sort['Власне значення'].values[0]
l2 = df_sort['Власне значення'].values[1]
print('l1 {:.6f}\nl2 {:.6f}'.format(l1, l2))

l_sum = np.sum(df_sort['Власне значення'])

delt_I = 1 - ((l1+l2)/l_sum)
delt_I *= 100
print('Відсоток втраченої інформації:\n{:.3} %'.format(delt_I))
```

```
l1 2.987789
l2 0.560719
Відсоток втраченої інформації:
12.2 %
```

9. Розрахувати принципові компоненти за допомогою вбудованої функції PCA з кількістю компонентів 2 з бібліотеки sklearn.

In [34]:

```
pca = PCA(n_components = 2)
XPCAreduced = pca.fit_transform(np.transpose(x_matrix))
```

10. Порівняти результати роботи власного алгоритму з результатами функції PCA з бібліотеки sklearn.

In [35]:

```
print ('Sklearn reduced X [:5]: \n', XPCAreduced[:5])
Sklearn reduced X [:5]:
 [[ 0.53000157 -0.81588358]
 [ 0.05648712 -1.0773472 ]
 [ 0.5744019  -0.4569081 ]
 [-2.38876911  0.74233881]
 [-0.32567545  0.07569688]]
```

In [36]:

```
x_new[:5]
```

Out[36]:

```
array([[-0.99582149, -1.28767501],
       [-0.31350845, -0.96865302],
       [-0.89581604, -0.86638657],
       [ 1.95293489,  0.46302455],
       [ 0.08385608, -0.40762341]])
```

In [37]:

```
print('x_scaled \t', np.shape(x_scaled))        # [ℓ × n]
print('x_new    \t', np.shape(x_new))           # [ℓ × nPC]
print('XPCAreduced \t', np.shape(XPCAreduced)) # [ℓ × nPC]
x_scaled         (100, 4)
x_new            (100, 2)
XPCAreduced      (100, 2)
```

11. Візуалізувати різницю в отриманих вибірках.

In [38]:

```
difference = (XPCAreduced-x_new)
```

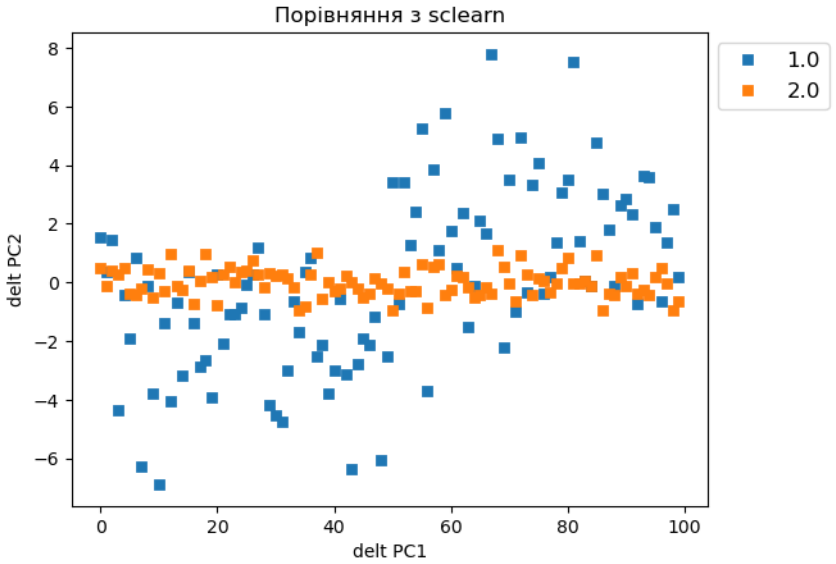
In [39]:

```
lab = dfIris2['target'].unique()
plt.plot(difference, 's', markersize = 5,
         #markerfacecolor='white',
         #markeredgecolor='gray',
         markeredgewidth = 2, label=lab)

plt.xlabel('delt PC1')
plt.ylabel('delt PC2')
plt.title('Порівняння з sclearn')
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[39]:

```
<matplotlib.legend.Legend at 0x220a2745700>
```

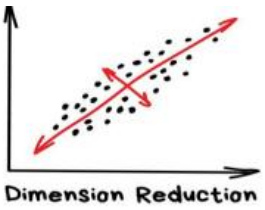


Редукування даних: метод SVD

Лабораторна робота 5

Редукування даних: метод SVD

Постановка задачі



Стиснення зображення з використанням методу SVD

1. Відцифрувати чорнобілий (за бажанням кольоровий) малюнок.
2. Провести розкладання SVD за допомогою вбудованої функції
3. Проаналізувати власні значення; побудувати гістограми перших 20 значень
4. Порахувати кількість втраченої інформації у відсотках за умови використання лише 1, 2, 3, ..., N компонентів
5. Побудувати графічно цю залежність
6. Відтворити малюнок для 1, 5, 15, ... головних компонент
7. Результати оформити у вигляді звіту

In [1]:

```
!pip install opencv-python
!pip install imutils
Requirement already satisfied: opencv-python in
c:\users\admin\anaconda3\lib\site-packages (4.7.0.72)
Requirement already satisfied: numpy>=1.17.0 in
c:\users\admin\anaconda3\lib\site-packages (from opencv-python) (1.22.4)
Requirement already satisfied: imutils in c:\users\admin\anaconda3\lib\site-
packages (0.5.4)
```

In [2]:

```
%matplotlib inline

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from matplotlib.image import imread
import os

# Import opencv
import cv2
```

In [3]:

```
import warnings
warnings.filterwarnings('ignore')
```

- **1**

In [4]:

```
name_img = 'beach.jpg'
img_orig = mpimg.imread(name_img)
```

```
imgplot = plt.imshow(img_orig)
```



- **2-3**

In [5]:

```
def fun_matrixs(name_img, components):
    # Завантажте вхідне зображення і перетворіть в чорнобіле
    img = cv2.imread(name_img, 0)

    original_shape = img.shape
    print('Розміри початкового зображення:', original_shape)

    U, S, V = np.linalg.svd(img)
    print('\nРозміри матриць отриманих з np.linalg.svd(...):\n U.shape, S.shape,
V.shape')
    print(U.shape, S.shape, V.shape)

    first = [min(np.shape(img))]
    #comps = [first, 200, 50, 15, 5, 1]
    comps = first + components
    print('\nСписок числа компонент:', comps)

    matr = []
    for i in range(len(comps)):
        low_rank = U[:, :comps[i]] @ np.diag(S[:comps[i]]) @ V[:comps[i], :]
        #print(np.shape(low_rank))
        matr.append(low_rank)

    return matr
```

In [6]:

```
components0 = [200, 100, 50, 25, 15, 10, 5, 4, 3, 2, 1]
matr0 = fun_matrixs(name_img, components0)
print(np.shape(matr0))
Розміри початкового зображення: (638, 960)

Розміри матриць отриманих з np.linalg.svd(...):
 U.shape, S.shape, V.shape
(638, 638) (638,) (960, 960)

Список числа компонент: [638, 200, 100, 50, 25, 15, 10, 5, 4, 3, 2, 1]
(12, 638, 960)
```

In [7]:

```
matr0_ = matr0[0]
covmat = np.cov(matr0_)
w, v = np.linalg.eig(covmat)
```

In [8]:

```
d2 = {'Власні значення': w[:20]}
df1 = pd.DataFrame(d2)
display(df1.head())
```

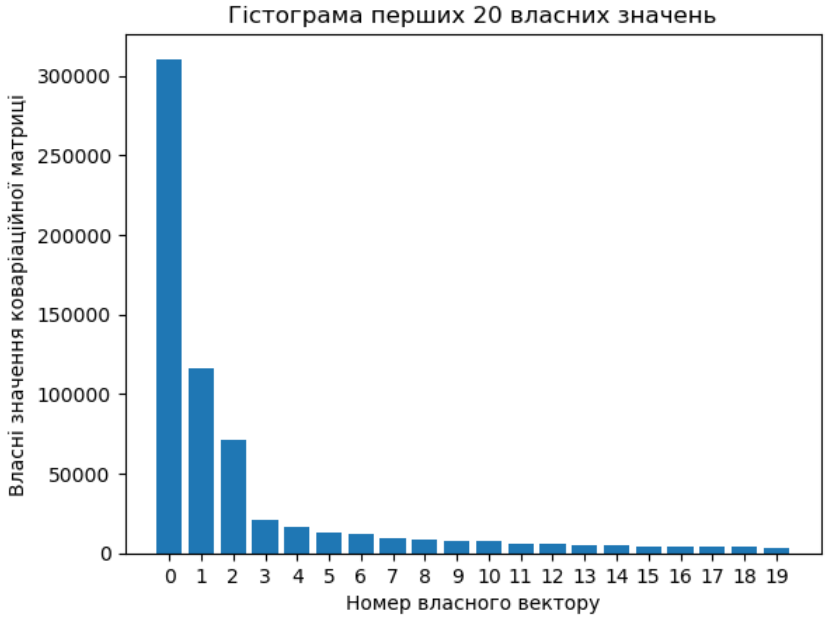
	Власні значення
0	310432.929103
1	116198.452046
2	71619.792069
3	21235.600645
4	16093.473605

In [9]:

```
numbers = np.arange(20)
plt.bar(numbers, height=df1['Власні значення'])
plt.xticks(numbers)
plt.title('Гістограма перших 20 власних значень')
plt.xlabel('Номер власного вектору')
plt.ylabel('Власні значення коваріаційної матриці')
```

Out[9]:

```
Text(0, 0.5, 'Власні значення коваріаційної матриці')
```



- **4**

In [10]:

```
def plot_components(name_img):
    # Завантажте вхідне зображення і перетвориіть в чорнобіле
    img = cv2.imread(name_img, 0)
    original_shape = img.shape
    print('Розміри початкового зображення:', original_shape)
```

```
    # Отримання svd
    U, S, V = np.linalg.svd(img)

    print('\nРозміри матриць отриманих з np.linalg.svd(...):\n U.shape, S.shape,
V.shape')
    print(U.shape, S.shape, V.shape)

    # зображення з різною кількістю компонентів
    first = min(np.shape(img))
    components = [first, 200, 50, 15, 5, 4, 3, 2, 1]
    print('\nСписок числа компонент:', components, '\n')

    plt.figure(figsize = (16, 8))
    for i in range(len(components)):
        low_rank = U[:, :components[i]] @ np.diag(S[:components[i]]) @
V[:components[i], :]
    return components

components = plot_components(name_img)
Розміри початкового зображення: (638, 960)

Розміри матриць отриманих з np.linalg.svd(...):
 U.shape, S.shape, V.shape
(638, 638) (638,) (960, 960)

Список числа компонент: [638, 200, 50, 15, 5, 4, 3, 2, 1]

<Figure size 1600x800 with 0 Axes>
```

In [11]:

```
x = components[::-1]
percentage = [(1 - x[i]/x[-1])*100 for i in range(len(x))]
print('Втрати інформації')
for i in percentage:
    print(round(i,3), '%')
Втрати інформації
99.843 %
99.687 %
99.53 %
99.373 %
99.216 %
97.649 %
92.163 %
68.652 %
0.0 %
```

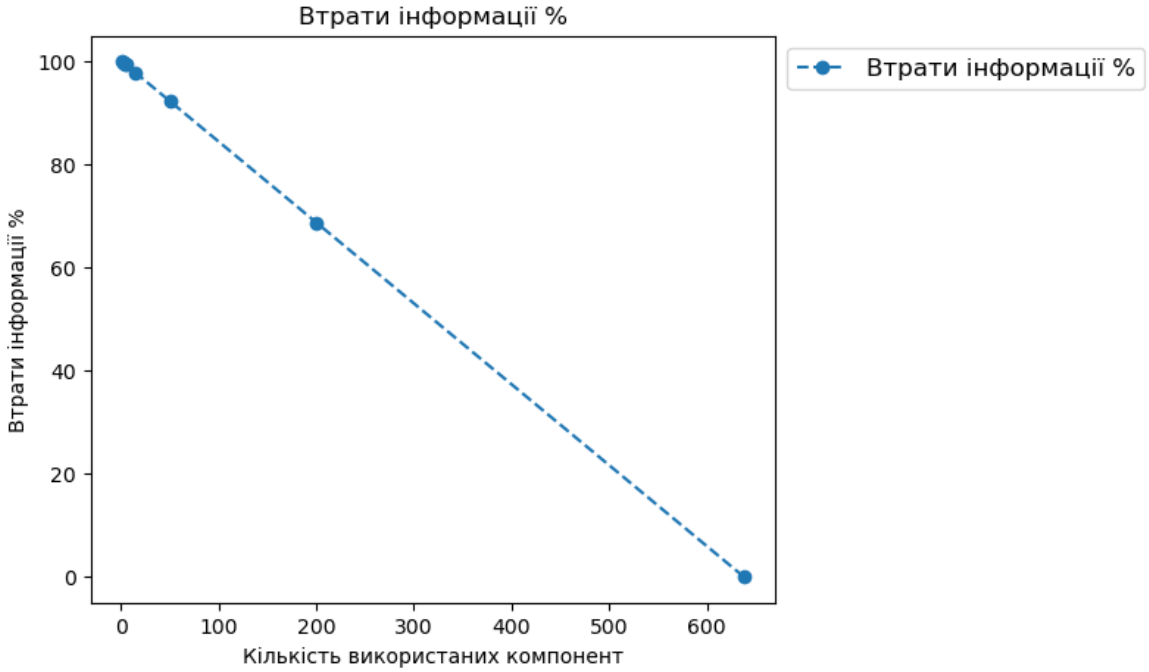
- **5**

In [12]:

```
plt.figure(figsize = (6, 5))
plt.plot(x, percentage, 'o--', label = 'Втрати інформації %')
plt.title('Втрати інформації %')
plt.xlabel('Кількість використаних компонент')
plt.ylabel('Втрати інформації %')
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[12]:

```
<matplotlib.legend.Legend at 0x1d82b7c14c0>
```



- **6**

In [13]:

```
def plot_n_comp(name_img):
    # Завантажте вхідне зображення і перетвориіть в чорнобіле
    img = cv2.imread(name_img, 0)
    original_shape = img.shape
    print('Розміри початкового зображення:', original_shape)

    # Отримання svd
    U, S, V = np.linalg.svd(img)

    print('\nРозміри матриць отриманих з np.linalg.svd(...):\n U.shape, S.shape,
V.shape')
    print(U.shape, S.shape, V.shape)

    # зображення з різною кількістю компонентів
    first = min(np.shape(img))
    components = [first, 300, 75, 15, 5, 1]
    print('\nСписок числа компонент:', components, '\n')

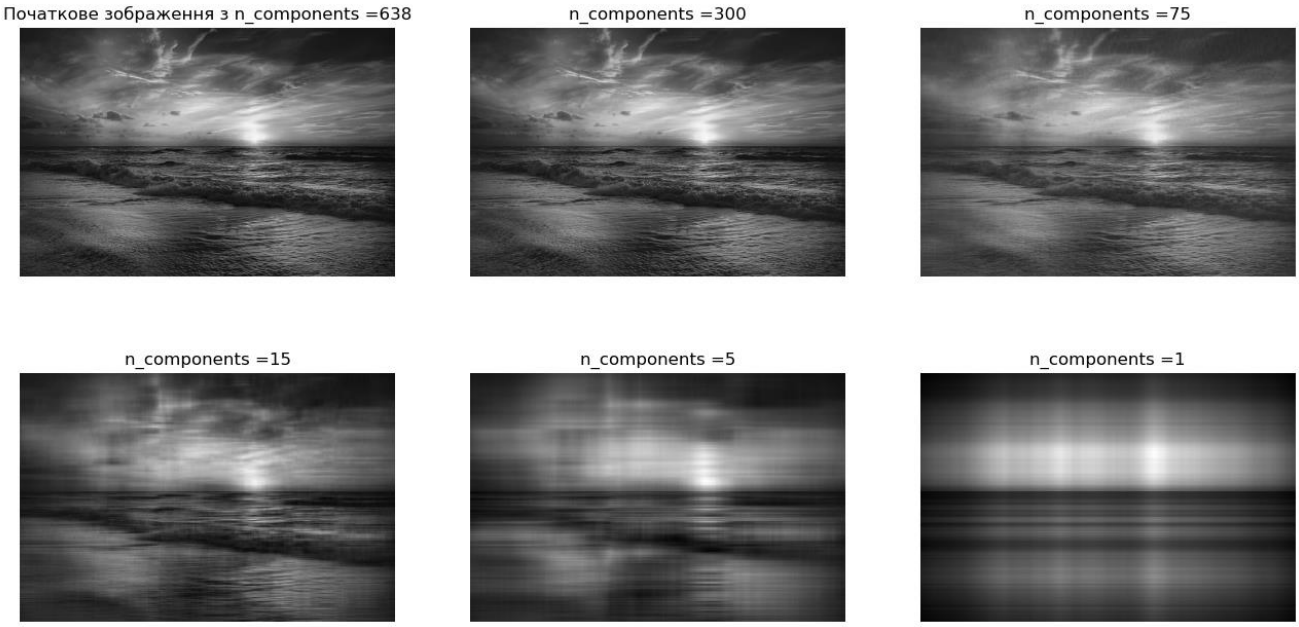
    plt.figure(figsize = (16, 8))
    for i in range(len(components)):
        low_rank = U[:, :components[i]] @ np.diag(S[:components[i]]) @
V[:components[i], :]
        if(i == 0):
            plt.subplot(2, 3, i+1), plt.imshow(low_rank, cmap = 'gray'),
plt.axis('off'), \
            plt.title("Початкове зображення з n_components =" +
str(components[i]))
        else:
            plt.subplot(2, 3, i+1), plt.imshow(low_rank, cmap = 'gray'),
plt.axis('off'), \
            plt.title("n_components =" + str(components[i]))

        #matr.append(low_rank)
        #print()

plot_n_comp(name_img)
Розміри початкового зображення: (638, 960)
```

```
Розміри матриць отриманих з np.linalg.svd(...):
 U.shape, S.shape, V.shape
(638, 638) (638,) (960, 960)

Список числа компонент: [638, 300, 75, 15, 5, 1]
```



Метод k-середніх (k-Means)

Лабораторна робота 6

5.2.3 Постановка задачі

Провести кластеризацію даних з використанням методу k-середніх.
Етапи розв'язання

1. Імпортувати/згенерувати розмічені/нерозмічені дані
2. У випадку кількості ознак $n > 2$ звести до $n = 2$ з використанням SVD та PCA
3. Провести кластеризацію методом k-means при фіксованому значенні кількості кластерів
4. Візуалізувати отримані результати
5. Встановити оптимальну кількість кластерів з використанням методів "метод ліктя" та "метод силуету". Порівняти отримані результатами з розміченими даними (за наявності)
6. Порівняти результати з k-means з sklearn
7. Оформити результати у вигляді звіту.

In [1]:

```
import warnings
warnings.filterwarnings('ignore')
```

In [2]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
%matplotlib inline
```

1. Імпортувати/згенерувати розмічені/нерозмічені дані

In [3]:

```
blobs = pd.read_csv('kmeans_blobs.csv')
colnames = list(blobs.columns[1:-1])
blobs.head()
```

Out[3]:

	ID	x	y	cluster
0	0	24.412	32.932	2
1	1	35.190	12.189	1
2	2	26.288	41.718	2
3	3	0.376	15.506	0
4	4	26.116	3.963	1

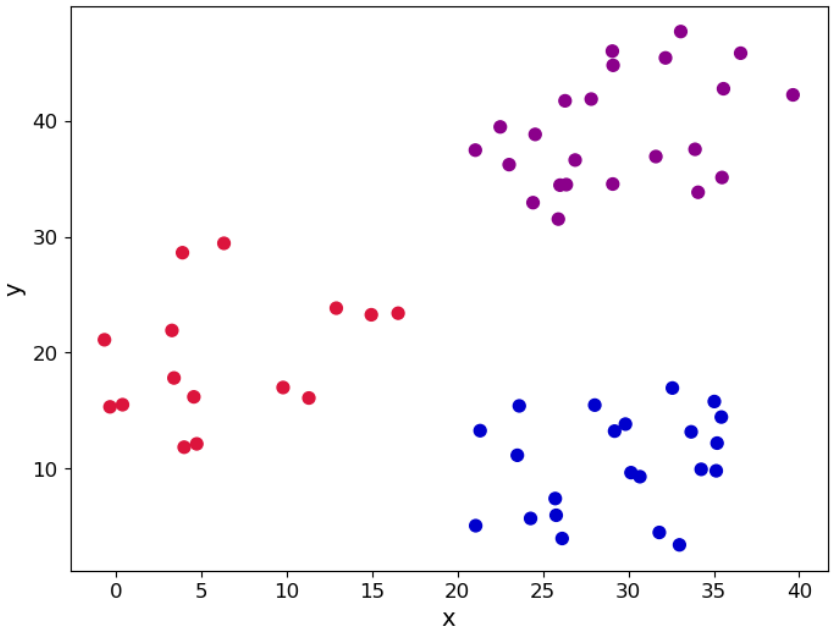
In [4]:

```
customcmap = ListedColormap(["crimson", "mediumblue", "darkmagenta"])


fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(x=blobs['x'], y=blobs['y'], s=50,
        c=blobs['cluster'].astype('category'),
        cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
```

```
plt.yticks(fontsize=12)

plt.show()
```



3. Провести кластеризацію методом k-means при фіксованому значенні кількості кластерів.

4. Візуалізувати отримані результати.

Кроки 1 і 2 - Визначення та ініціювання центроїдів.

In [5]:

```
def initiate_centroids(k, dset):
    '''
    Вибрати k точок даних як центроїди
    k: кількість центроїдів
    dset: pandas dataframe
    '''
    centroids = dset.sample(k)
    return centroids


np.random.seed(42)
k=3
df = blobs[['x','y']]
centroids = initiate_centroids(k, df)
centroids
```

Out[5]:

	x	y
0	24.412	32.932
5	25.893	31.515
36	26.878	36.609

Крок 3 - Розрахуйте відстань.

In [6]:

```
def rsserr(a,b):
    # Обчислити квадратний корінь з суми похибок. a та b - numpy масиви
    return np.square(np.sum((a-b)**2))
```

Крок 4 - Призначте центроїди.

In [7]:

```
def centroid_assignation(dset, centroids):
    '''
    Маючи датафрейм `dset` і набір `centroids`, ми призначаємо кожній
    точці даних у `dset` призначаємо центроїд.
    - dset - pandas датафрейм зі спостереженнями
    - centroids - фрейм даних з центроїдами
    '''
    k = centroids.shape[0]
    n = dset.shape[0]
    assignation = []
    assign_errors = []

    for obs in range(n):
        # Похибка оцінки
        all_errors = np.array([])
        for centroid in range(k):
            err = rsserr(centroids.iloc[centroid, :], dset.iloc[obs,:])
            all_errors = np.append(all_errors, err)

        # найближчий центроїд і помилку
```

```
        nearest_centroid =  np.where(all_errors==np.amin(all_errors))[0].tolist()[0]
        nearest_centroid_error = np.amin(all_errors)

        assignation.append(nearest_centroid)
        assign_errors.append(nearest_centroid_error)

    return assignation, assign_errors
```

In [8]:

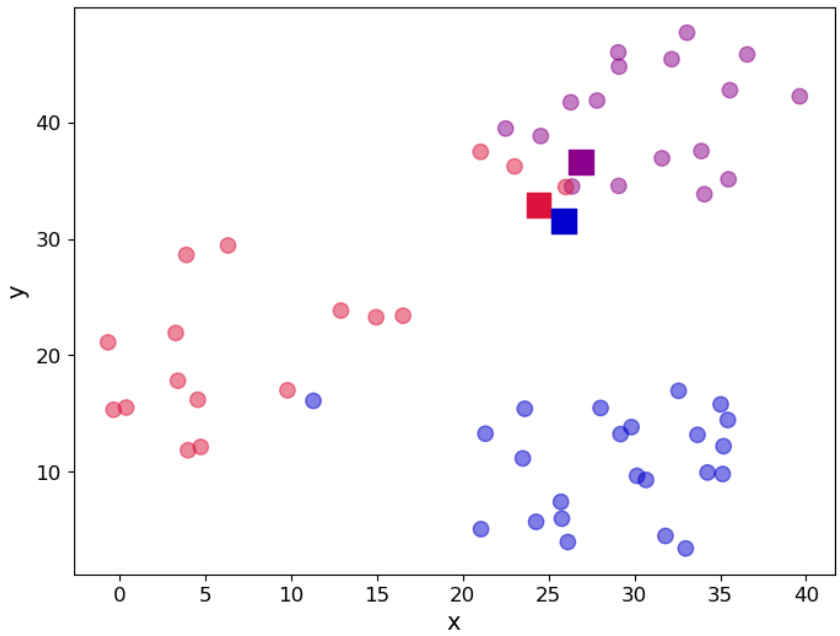
```
df['centroid'], df['error'] = centroid_assignation(df, centroids)
df.head()
```

Out[8]:

	x	y	centroid	error
0	24.412	32.932	0	0.000000
1	35.190	12.189	1	211534.211314
2	26.288	41.718	2	699.601495
3	0.376	15.506	0	776856.744109
4	26.116	3.963	1	576327.599678

In [9]:

```
fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(df.iloc[:,0], df.iloc[:,1],  marker = 'o',
          c=df['centroid'].astype('category'),
          cmap = customcmap, s=80, alpha=0.5)
plt.scatter(centroids.iloc[:,0], centroids.iloc[:,1],
          marker = 's', s=200, c=[0, 1, 2],
          cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



In [10]:

```
print("Сумарна помилка is {0:.2f}".format(df['error'].sum()))
```

Сумарна помилка is 11927659.01

Крок 5 - Оновлення розташування центроїда.

In [11]:

```
centroids = df.groupby('centroid').agg('mean').loc[:, colnames].reset_index(drop = True)
centroids
```

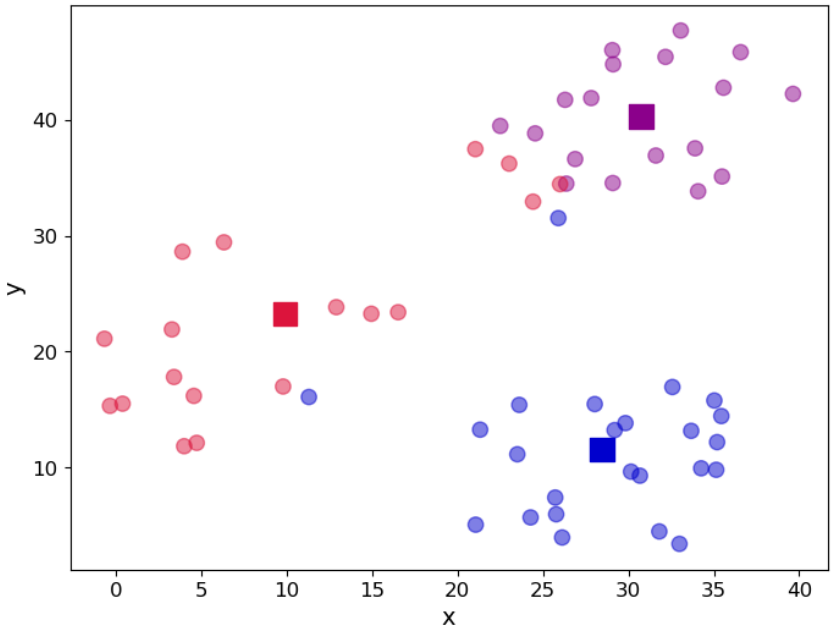
Out[11]:

	x	y
0	9.889444	23.242611
1	28.435750	11.546250
2	30.759333	40.311167

In [12]:

```
fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(df.iloc[:,0], df.iloc[:,1],  marker = 'o',
        c=df['centroid'].astype('category'),
        cmap = customcmap, s=80, alpha=0.5)
plt.scatter(centroids.iloc[:,0], centroids.iloc[:,1],
        marker = 's', s=200,
        c=[0, 1, 2], cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
```

```
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



Крок 6 - Повторіть кроки 3-5.

In [13]:

```
def kmeans(dset, k=2, tol=1e-4):
    '''
    K-means реалізовано для набору
    `dset`:  датафрей зі спостереженнями
    `k`: кількість кластерів, за замовчуванням k=2
    `tol`: допуск=1E-4
    '''
    # Давайте працювати в копії, щоб не зіпсувати оригінал
    working_dset = dset.copy()

    # Визначимо деякі змінні для зберігання помилки,
    # сигналу зупинки та лічильник для ітерацій

    err = []
    goahead = True
```

```
    j = 0

    # Крок 2: Ініціюйте кластери, визначивши центроїди
    centroids = initiate_centroids(k, dset)

    while(goahead):
        # Крок 3 і 4 - Призначення центроїдів і розрахунок похибки
        working_dset['centroid'], j_err = centroid_assignation(working_dset, centroids)
        err.append(sum(j_err))

        # Крок 5 - Оновлення положення центроїда
        centroids = working_dset.groupby('centroid').agg('mean').reset_index(drop = True)

        # Крок 6 - Перезапустіть ітерацію
        if j>0:
            # Чи є похибка меншою за допуск (1E-4)
            if err[j-1]-err[j]<=tol:
                goahead = False
        j+=1

    working_dset['centroid'], j_err = centroid_assignation(working_dset, centroids)
    centroids = working_dset.groupby('centroid').agg('mean').reset_index(drop = True)
    return working_dset['centroid'], j_err, centroids
```

In [14]:

```
np.random.seed(42)
df['centroid'], df['error'], centroids =  kmeans(df[['x','y']], 3)
df.head()
```

Out[14]:

	x	y	centroid	error
0	24.412	32.932	2	3767.568743
1	35.190	12.189	1	1399.889001

	x	y	centroid	error
2	26.288	41.718	2	262.961097
3	0.376	15.506	0	2683.086425
4	26.116	3.963	1	2723.650198

In [15]:

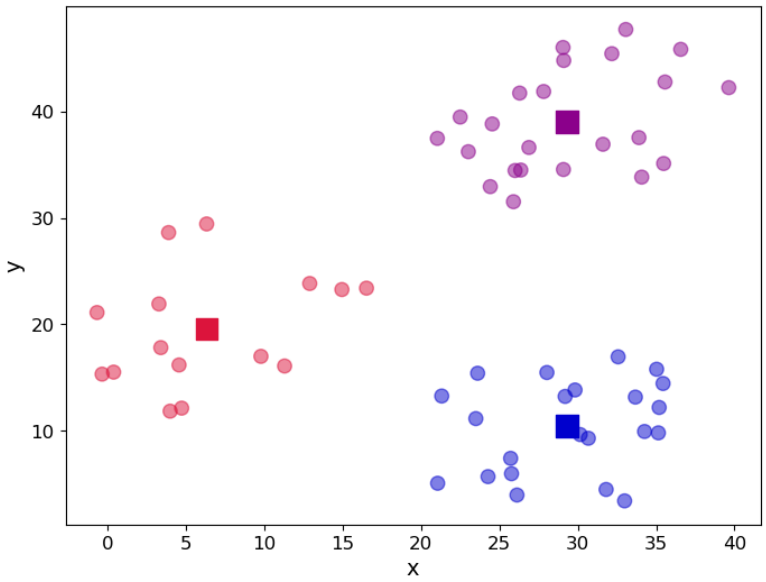
```
centroids
```

Out[15]:

	x	y
0	6.322867	19.559800
1	29.330864	10.432409
2	29.304957	39.050783

In [16]:

```
fig, ax = plt.subplots(figsize=(8, 6))
plt.scatter(df.iloc[:,0], df.iloc[:,1],  marker = 'o',
        c=df['centroid'].astype('category'),
        cmap = customcmap, s=80, alpha=0.5)
plt.scatter(centroids.iloc[:,0], centroids.iloc[:,1],
        marker = 's', s=200, c=[0, 1, 2],
        cmap = customcmap)
ax.set_xlabel(r'x', fontsize=14)
ax.set_ylabel(r'y', fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.show()
```



Отримали 3 групи.У цьому конкретному прикладі дані такі, що різниця між групами є чіткою. Однак, нам може не так пощастити в кожному випадку. Тому питання про те, скільки груп існує, залишається відкритим. Ми можемо використати графік, який допоможе нам мінімізувати похибку, подивившись на запуск алгоритму з послідовністю і шукати "лікоть" на графіку, який вказує на достатню кількість кластерів для використання:

5. Встановити оптимальну кількість кластерів з використанням методів “метод ліктя” / “метод силуету”. Порівняти отримані результатами з розміченими даними (за наявності).

In [17]:

```
n = 10

err_total = []


df_elbow = blobs[['x','y']]


for i in range(n):

    working_dset, my_errs, centroids = kmeans(df_elbow, i+1)

    err_total.append(sum(my_errs))


fig, ax = plt.subplots(figsize=(7, 4))

plt.plot(range(1,n+1), err_total, linewidth=3, marker='o')

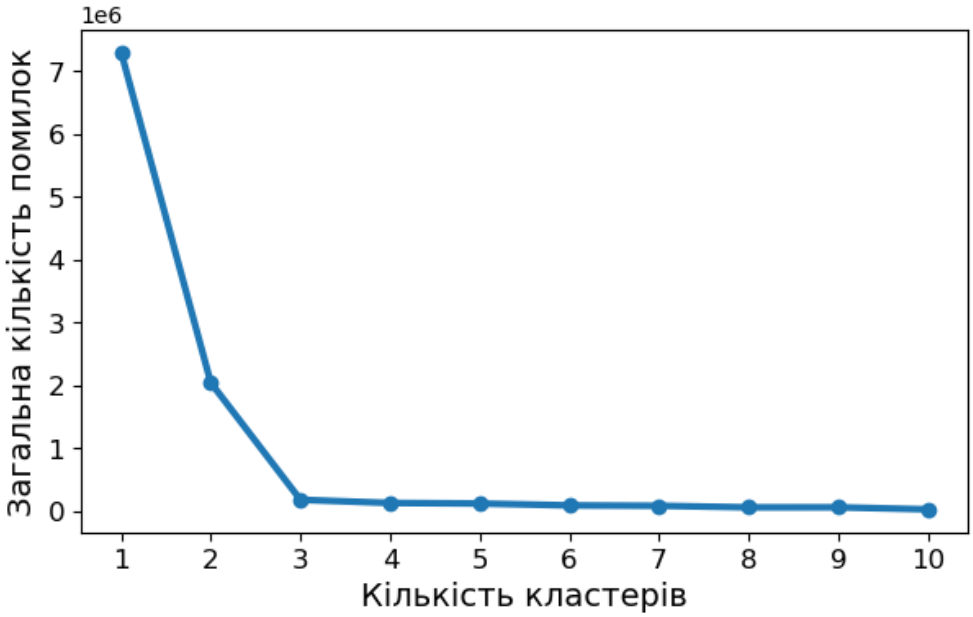
ax.set_xlabel(r'Кількість кластерів', fontsize=14)

ax.set_ylabel(r'Загальна кількість помилок', fontsize=14)

plt.xticks(range(1, n+1), fontsize=12)

plt.yticks(fontsize=12)
```

```
plt.show()
```



Тепер ми можемо застосувати "правило ліктя", яке є евристичним прийомом, що допоможе нам визначити кількість кластерів. Якщо уявити лінію, зображену вище, як руку, то "лікоть" - це точка перегину. У цьому випадку "лікоть" знаходиться між 2 і 4 кластерами, що вказує нам на те, що вибір 3 є правильним.

Порівняти отримані результатами з розміченими даними (за наявності).

In [18]:

```
blobs = pd.read_csv('kmeans_blobs.csv')

blobs.head()
```

Out[18]:

	ID	x	y	cluster
0	0	24.412	32.932	2
1	1	35.190	12.189	1
2	2	26.288	41.718	2
3	3	0.376	15.506	0
4	4	26.116	3.963	1

In [19]:

```
sns.scatterplot(x = blobs['x'], y = blobs['y'], hue = blobs['cluster'])

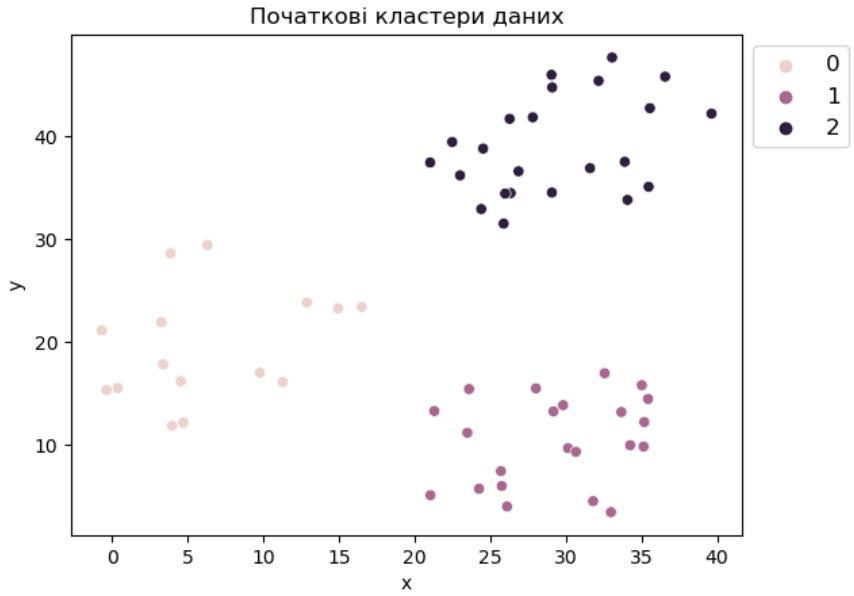
plt.xlabel("x")

plt.ylabel("y")

plt.title("Початкові кластери даних" )
```

```
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})

plt.show()
```



6. Порівняти результати з k-means з sklearn

In [20]:

```
from sklearn.cluster import KMeans
```

In [21]:

```
data = pd.read_csv('kmeans_blobs.csv')

data.head()
```

Out[21]:

	ID	x	y	cluster
0	0	24.412	32.932	2
1	1	35.190	12.189	1
2	2	26.288	41.718	2
3	3	0.376	15.506	0
4	4	26.116	3.963	1

In [22]:

```
data_xy = data[['x', 'y']]

data_xy.head()
```

Out[22]:

	x	y
0	24.412	32.932
1	35.190	12.189
2	26.288	41.718
3	0.376	15.506
4	26.116	3.963

In [23]:

```
inertias = []

n=10

for i in range(1,n+1):
    kmeans = KMeans(n_clusters=i)
    kmeans.fit(data_xy)
    inertias.append(kmeans.inertia_)


plt.plot(range(1,11), inertias, marker='o')

plt.title('Метод ліктя')

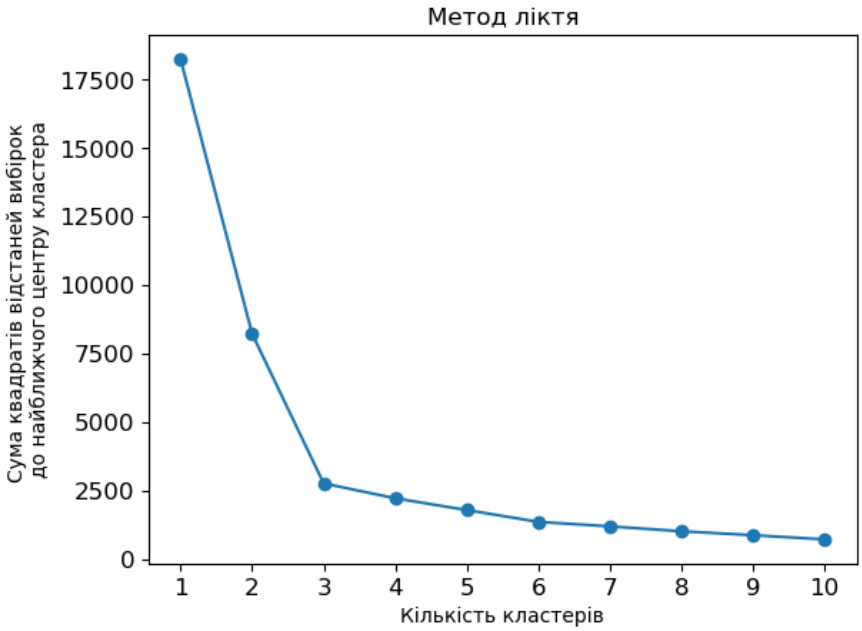
plt.xlabel('Кількість кластерів')

plt.ylabel('Сума квадратів відстаней вибірок \nдо найближчого центру кластера')

plt.xticks(range(1, n+1), fontsize=12)

plt.yticks(fontsize=12)

plt.show()
```



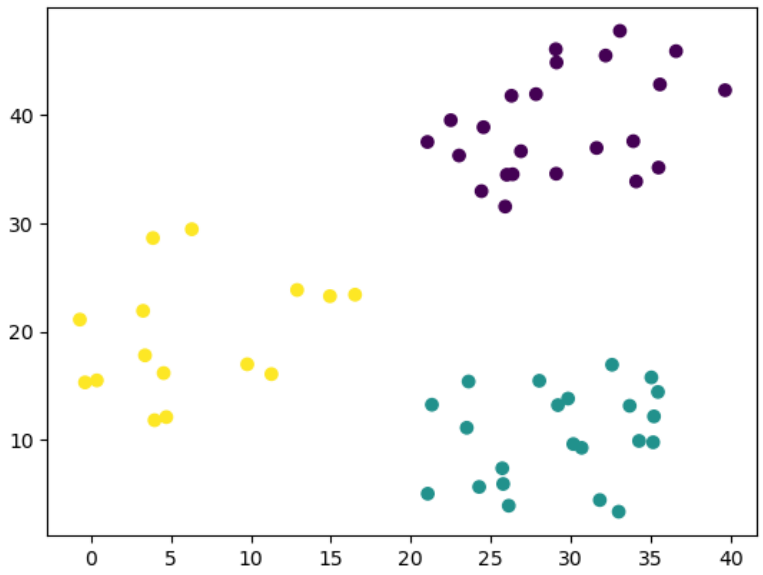
In [24]:

```
kmeans = KMeans(n_clusters=3)

kmeans.fit(data_xy)


plt.scatter(data_xy['x'], data_xy['y'], c=kmeans.labels_)

plt.show()
```



5.3.7 Постановка задачі

Провести кластеризацію даних з використанням методу DBSCAN.
Етапи розв'язання

1. Імпортувати/згенерувати розмічені/нерозмічені дані
2. У випадку кількості ознак $n > 2$ звести до $n = 2$ з використанням SVD та PCA
3. Зафіксувати кількість сусідів m, та побудувати K-Distance Graph та визначити оптимальне значення щільності ϵ
4. Провести кластеризацію методом DBSCAN при обраних параметрах m та ϵ
5. Візуалізувати отримані результати
6. Порівняти результати з DBSCAN з sklearn
7. Оформити результати у вигляді звіту.

In [1]:

```
import math

import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt
```

In [2]:

```
from sklearn.preprocessing import normalize

from sklearn.cluster import DBSCAN

from scipy.spatial import distance

from sklearn.neighbors import NearestNeighbors
```

1

In [3]:

```
# генеруємо дані

def make_circle(r, n, noise = 30, seed = 1234):

    np.random.seed(seed)
```

```
    return [(math.cos(2*math.pi/n*x)*r+np.random.normal(-noise,noise),
math.sin(2*math.pi/n*x)*r+np.random.normal(-noise,noise)) for x in range(1,n+1)]


small_circle = make_circle(100, 300, 10)

medium_circle = make_circle(300, 700, 20)

big_circle = make_circle(500, 1000, 30)


noise = [(np.random.randint(-600,600),np.random.randint(-600,600)) for i in range(300)]
```

In [4]:

```
def arrray_to_df(arr, i):
    df = pd.DataFrame(arr)
    df['cluster'] = str(i)
    return df


data = [arrray_to_df(arr, i) for i, arr in enumerate([small_circle, medium_circle, big_circle, noise])]

data = pd.concat(data)

data.columns = ['x', 'y', 'cluster']
```

In [5]:

```
plt.scatter(data['x'], data['y'], label='data')

plt.xlabel('x')

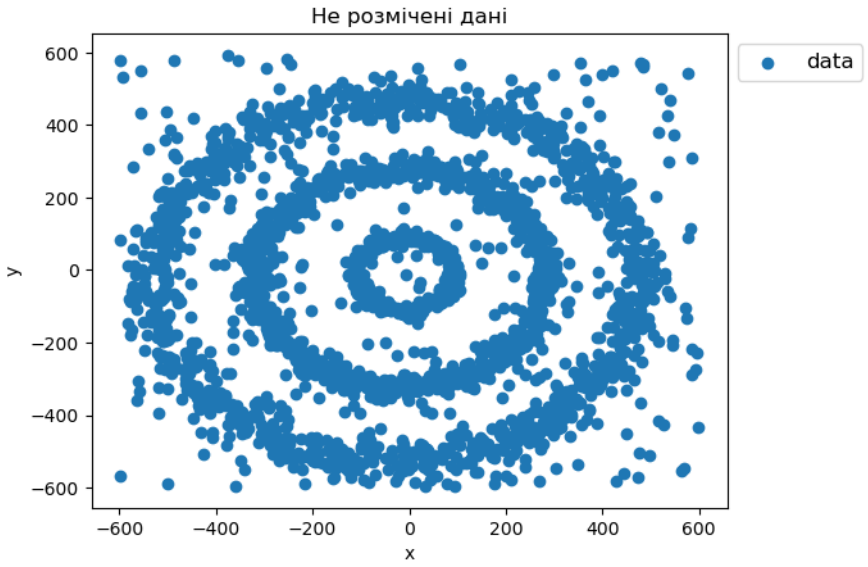
plt.ylabel('y')

plt.title("Не розмічені дані")

plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[5]:

```
<matplotlib.legend.Legend at 0x135e08a15b0>
```



In [6]:

```
plt.rcParams['figure.figsize'] = [8,6]

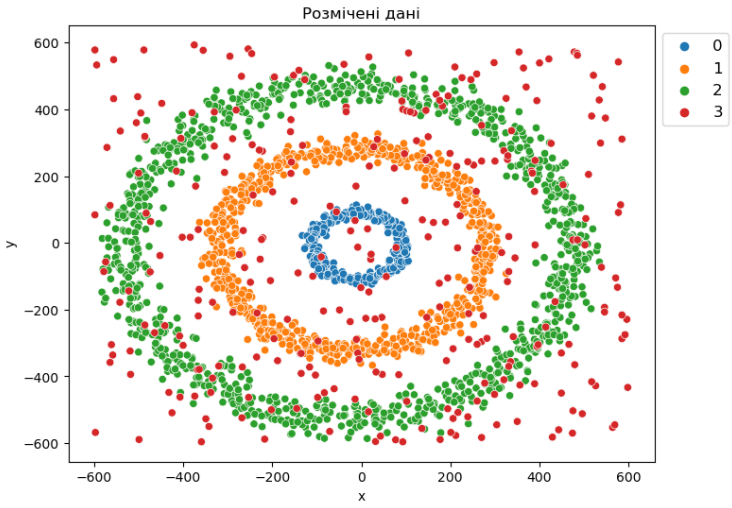
sns.scatterplot(data = data, x = 'x',y = 'y',hue = 'cluster')

plt.title("Розмічені дані")

plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[6]:

```
<matplotlib.legend.Legend at 0x135e141f1f0>
```



3

In [7]:

```
k = 2

data_nn = data.copy()[['x', 'y']]


nearest_neighbors = NearestNeighbors(n_neighbors=k)
```

```
neighbors = nearest_neighbors.fit(data_nn)
distances, indices = neighbors.kneighbors(data_nn)
distances = np.sort(distances, axis=0)

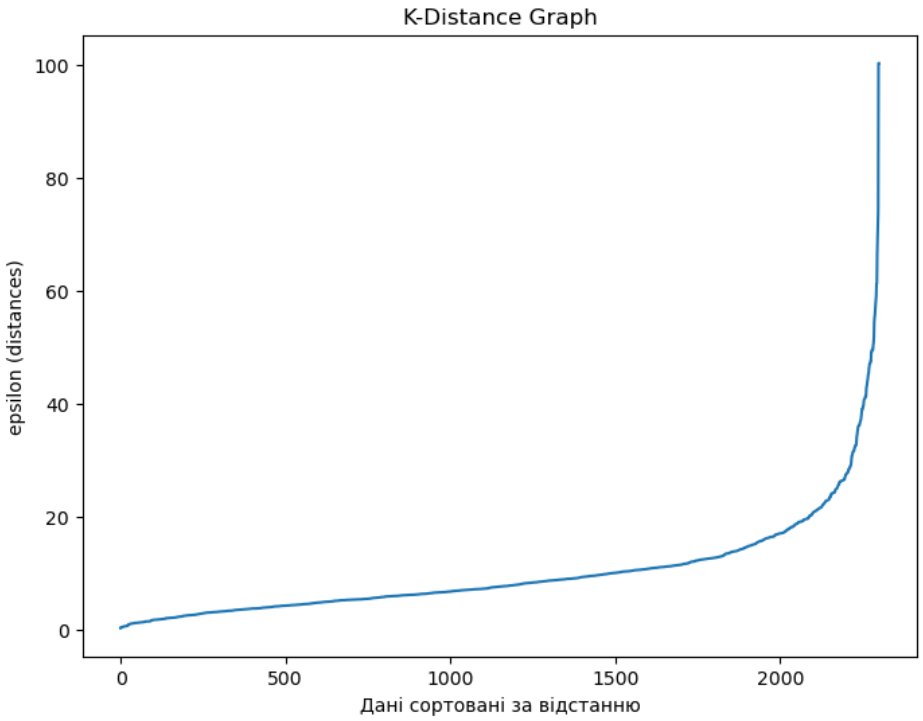
distances = distances[:,1]

i = np.arange(len(distances))

sns.lineplot(x = i, y = distances)
plt.xlabel("Дані сортовані за відстанню")
plt.ylabel("epsilon (distances)")
plt.title('K-Distance Graph')
```

Out[7]:

```
Text(0.5, 1.0, 'K-Distance Graph')
```



4

In [8]:

```
class dbscan:

    def __init__(self, epsilon = None, min_samples = None, distance = 'euclidean', normalize = False):
        self.epsilon = epsilon
```

```
    self.min_samples = min_samples
    self.distance = distance
    self.normalize = normalize

def find_distance(self, x, type = 'euclidean'):
    """
    Finds distance between numpy arrays.
    """
    return distance.squareform(distance.pdist(x, type))

def normalization(self, x):
    return (x-np.min(x))/(np.max(x) - np.min(x))

def find_neighbors(self, x):
    return np.where(x <= self.epsilon)[0]

def expand_cluster(self, neighbors, x, cluster, labels):

# Iterate over each neighbor
    for neighbor in neighbors:
  # Check that is not assigned
        if labels[neighbor] == 0:
            # Find neighbors
            neighbor_neighbors = self.find_neighbors(x[neighbor])
            # Check if is core
            if len(neighbor_neighbors) >= self.min_samples:
                labels[neighbor] = cluster
                # For each neighbor in neighbors, expand cluster
                labels = self.expand_cluster(neighbor_neighbors, x, cluster, labels)
    return labels

def fit(self, x):
```

```
    """
    Given a reference point and comparison points and a distance function, returns the index of the neighbors.
    """
    # Do normalization
    if self.normalize:
        x = self.normalization(x)

    # Find distance
    dist_matrix = self.find_distance(x, self.distance)

    # Initialize cluster
    cluster = 1
    n_obs = x.shape[0]
    labels = np.zeros(n_obs)

    for i in range(n_obs):

        # If value not assigned
        if labels[i] == 0:

            # Find neighbors
            neighbors = self.find_neighbors(dist_matrix[i])

            # Check if neighbors > min_samples (self included as neighbor)
            if len(neighbors) > self.min_samples:

                # If observation is not assigned --> Assign to cluster
                if labels[i] == 0:
                    labels[i] = cluster

                # Expand cluster on neighbors
```

```
            labels = self.expand_cluster(neighbors, dist_matrix, cluster, labels)

            # Go to next cluster
            cluster = cluster + 1

        return labels
```

In [9]:

```
data['dbscan_custom'] = dbscan(epsilon=32, min_samples=5).fit(data[['x', 'y']].to_numpy())
```

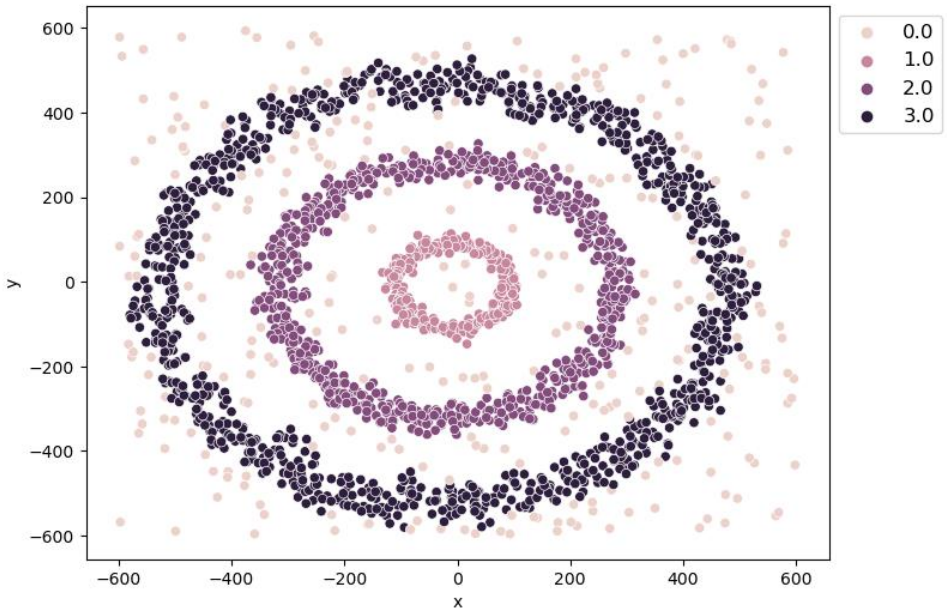
5

In [10]:

```
sns.scatterplot(data = data, x = 'x', y = 'y', hue = 'dbscan_custom')
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[10]:

```
<matplotlib.legend.Legend at 0x135e10e8bb0>
```



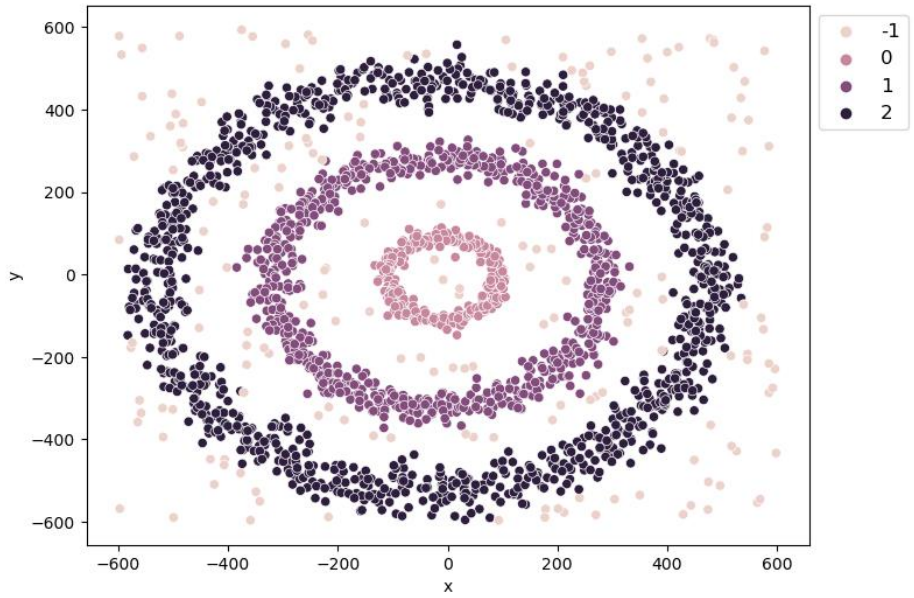
6

In [11]:

```
data2 = data.copy()
data2['dbscan2'] = DBSCAN(eps=32, min_samples=5).fit_predict(data[['x', 'y']])
sns.scatterplot(data = data2, x = 'x', y = 'y', hue = 'dbscan2')
plt.legend(bbox_to_anchor=(1,1), loc="upper left", prop={'size': 12})
```

Out[11]:

<matplotlib.legend.Legend at 0x135e1119880>



Отже, результати ідентичні.

Асоціації та пошук правил: метод Apriori

Лабораторна робота 8

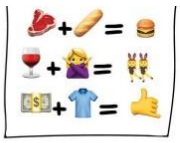
Пошук асоціативних правил з використанням методу Apriori

Імпортувати дані для методу Apriori

1. Фіксуємо значення мінімальної підтримки та мінімальної впевненості
2. Використовуємо вбудований метод apriori з бібліотеки apyori
3. Отримуємо правила асоціації
4. Сортуємо правила за зменшенням впевненості
5. Виводимо правила на екран (файл)
6. Результати оформити у вигляді звіту

Асоціації та пошук правил: метод Apriori

Приклад подання результатів



	Left hand side	Right hand side	Support	Confidence
11	pasta	shrimp	0.005067	0.322034
6	fromage blanc	honey	0.003333	0.245098
9	light cream	olive oil	0.003200	0.205128
5	fresh tuna	honey	0.004000	0.179641
7	ground beef	herb & pepper	0.016000	0.162822
10	olive oil	whole wheat pasta	0.008000	0.121704
0	brownies	cottage cheese	0.003467	0.102767
4	fresh bread	tomato juice	0.004267	0.099071
1	chicken	light cream	0.004533	0.075556
3	escalope	pasta	0.005867	0.073950

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from apyori import apriori
```

In [2]:

```
!pip install apyori
Requirement already satisfied: apyori in c:\users\admin\anaconda3\lib\site-
packages (1.1.2)
```

In [3]:

```
dataset = pd.read_csv("Market_Basket_Optimisation.csv")
dataset
```

Out[3]:

	shrimp	almonds	avocado	vegetables mix	green grapes	whole wheat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie	spinach	olive oil
0	burgers	meatballs	eggs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	chutney	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	turkey	avocado	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	mineral water	milk	energy bar	whole wheat rice	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	low fat yogurt	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
7495	butter	light mayo	fresh bread	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7496	burgers	frozen vege	eggs	french fries	magazines	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

	shrimp	almonds	avocado	vegetables mix	green grapes	whole wheat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie	spinach	olive oil
		tables																		
7497	chicken	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7498	escalope	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7499	eggs	frozen smoothie	yogurt cake	low fat yogurt	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

7500 rows × 20 columns

In [4]:

```
len(dataset)
```

Out[4]:

```
7500
```

In [5]:

```
data_new = dataset.drop(range(3500,len(dataset)))
data_new
```

Out[5]:

	shrimp	almonds	avocado	vegetables mix	green grapes	whole wheat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie	spinach	olive oil
0	burgers	meatballs	eggs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	chutney	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	turkey	avocado	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

	shrimp	almonds	avocado	vegetables mix	green grapes	whole wheat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie	spinach	olive oil
3	mineral water	milk	energy bar	whole wheat rice	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	low fat yogurt	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
3495	burgers	salmon	vegetables mix	carrots	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3496	tomatoes	rice	oil	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3497	herb & pepper	ground beef	salmon	spinach	honey	protein bar	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3498	chocolate	red wine	tomato sauce	butter	chicken	salmon	french fries	salt	mint	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3499	cookies	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

3500 rows × 20 columns

In [6]:

```
transactions = []
for i in range(0, len(data_new)):
    transactions.append([str(data_new.values[i,j]) for j in range(0,20)])
```

In [7]:

```
rules = apriori(transactions = transactions, min_support = 0.003, min_cinfidence
= 0.2, \
                min_lift = 3, min_length = 2, max_length = 2)
```

In [8]:

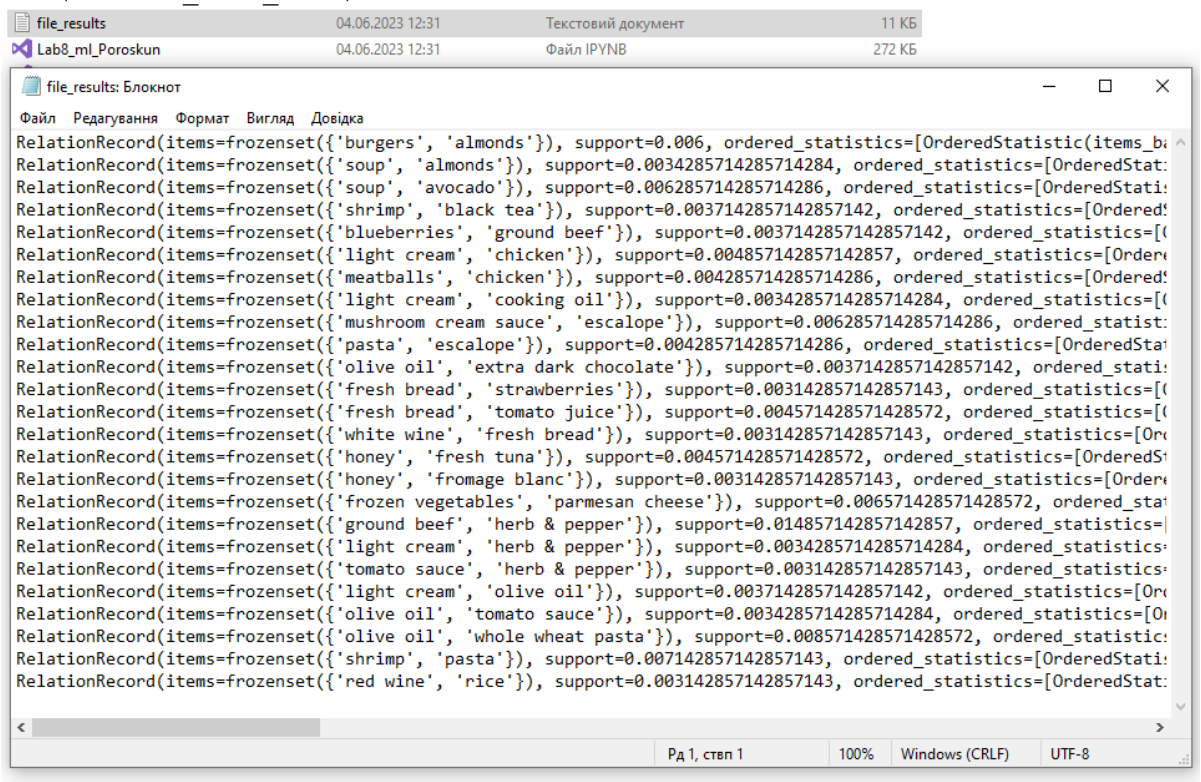
```
results = list(rules)
#results
```

In [9]:

```
with open("file_results.txt", 'w') as f:
    for s in results:
        f.write(str(s) + '\n')

with open("file_results.txt", 'r') as f:
    results_from_file = [line.rstrip('\n') for line in f]

#print(results_from_file)
```



In [10]:

```
def inspect(results):
    lhs         =[tuple(result[2][0][0])[0] for result in results]
    rhs         =[tuple(result[2][0][1])[0] for result in results]
    supports    =[result[1] for result in results]
    confidences =[result[2][0][2] for result in results]
    lifts        =[result[2][0][3] for result in results]
    return list (zip(lhs, rhs, supports, confidences, lifts))
```

In [11]:

```
resultsinDataFrame = pd.DataFrame(inspect(results), \
                                  columns = ["Left hand side", "Right hand
side", "Support", "Confidence", "Lift"])
#resultsinDataFrame
```

In [12]:

```
resultsinDataFrame.drop('Lift', axis = 1).nlargest(n = 10, columns =
"Confidence")
```

Out[12]:

	Left hand side	Right hand side	Support	Confidence
23	pasta	shrimp	0.007143	0.423729
4	blueberries	ground beef	0.003714	0.371429
0	almonds	burgers	0.006000	0.300000

	Left hand side	Right hand side	Support	Confidence
10	extra dark chocolate	olive oil	0.003714	0.295455
3	black tea	shrimp	0.003714	0.282609
15	fromage blanc	honey	0.003143	0.275000
20	light cream	olive oil	0.003714	0.209677
14	fresh tuna	honey	0.004571	0.192771
2	avocado	soup	0.006286	0.174603
1	almonds	soup	0.003429	0.171429

Висновки

У ході виконання роботи було проведено аналіз 8 тем машинного навчання, завданнями яких була класифікація, кластеризація, зменшення розмірності даних та пошук асоціативних правил. Частина алгоритмів була розроблена і результати були подібні до вже вбудованих методів. Тому вони розроблені правильно. Також були розглянуті результати виконання вбудованих функцій.